Supplementary Material: Verified Optimization, Export and Evaluation of a Translation from Real-Time Requirements to Timed Büchi Automata

Elie Fares, Jean-Paul Bodeveix, Azar Salami, Hussain Al-Aqrabi

Abstract

This supplement contains the proof details and the material that the article summarizes: the definitions and proofs of the verified passes, the general split construction with its two policies, the conditions and proof arguments of the two certificate-checked passes, the tool details (speedups, input contract, build and artifact layout), the translator-independent synthetic evaluation, the generated-group results, and the additional evaluation material: the TChecker and generation-time tables, the UPPAAL models, the differential test, the seeded-defect study with its Rocq rejections, and the random-formula and export-policy results. References to the article are marked "article"; all other references and numbers are to this supplement.

1. Verified passes: definitions and proofs

1.1. Location invariants on the trace model

A location invariant maps every clock x either to an upper bound $M \in \mathbb{R}$, standing for the constraint $x \leq M$, or to $+\infty$ (no constraint). We interpret automata with invariants on the extended words of article Section 2, as we do for automata without invariants. Along a run r, the automaton stays in location $r(i)$ between the events $i-1$ and i, and the invariant of $r(i)$ must hold during the whole stay, that is, at the clock values $\mathsf{val}_\rho(i,x)-\tau$ for every delay $\tau \in [0, \Delta_{i-1}]$, where $\Delta_{i-1} = t_i - t_{i-1}$ (and at the values $\mathsf{val}_\rho(0,x)$ for the initial location). This is the condition of article Section 2, evaluated at the event that ends the stay. Since clocks increase while time elapses, an invariant made of upper bounds holds during the stay if and only if it holds at the event that ends it (lemma `inv_during_iff_at_event`). For the

two-sided invariants of Section 1.3, a lower bound $x \geq m$ must likewise hold during the whole stay, hence at its beginning.

1.2. Invariant synthesis and backward propagation

In the exported timed automaton, upper-bounded guards need invariants so that time does not elapse past all useful outgoing transitions. We compute location invariants from outgoing upper bounds and propagate successor requirements backward, using location invariants to constrain admissible time elapse as in standard UPPAAL models [1].

Upper bound of a guard.. For a conjunctive guard g and a clock x, let $ub_x(g)$ be the smallest bound m of the constraints $x \leq m$, $x < m$, and $x = m$ occurring in g, and $+\infty$ if g contains none. Other constraints are ignored. A valuation satisfying g satisfies $x \leq ub_x(g)$.

Invariant synthesis.. For a location ℓ with outgoing transitions $\ell \xrightarrow{g_i, Z_i} \ell_i$, $i = 1, \ldots, m$, the synthesized invariant is

$$Inv(\ell)(x) = \max_{1 \leq i \leq m} ub_x(g_i),$$

with $Inv(\ell)(x) = +\infty$ when ℓ has no outgoing transition. Thus a clock that some outgoing guard does not bound does not appear in the invariant. The bound is always non-strict, even when it comes from a strict guard $x < m$: the invariant only needs to be implied by the guard of the next transition, and strictness is kept in the guards themselves.

Theorem 1.1 (Invariant synthesis)**.** *For every TBA $\mathcal{A}$, the automaton $\mathcal{A}$ with the synthesized invariants accepts the same timed words as $\mathcal{A}$.*

Proof. Adding invariants can only remove runs. Conversely, let r be an accepting run on ρ. At every event i, the transition taken from $r(i)$ is enabled, so its guard holds at the values $\mathsf{val}_\rho(i, x)$, which are therefore at most $Inv(r(i))(x)$. By the lemma `inv_during_iff_at_event`, the invariant of $r(i)$ holds during the whole stay. (Rocq: `add_invariants_correct`.) □

Backward propagation.. One propagation step strengthens every transition $\ell \xrightarrow{g, Z} \ell'$ with the invariant of its target after the resets:

$$g := g \wedge Inv(\ell')[Z := 0],$$

where $Inv(\ell')[Z := 0]$ replaces every bound $x \leq M$ on a reset clock $x \in Z$ by the constant constraint $0 \leq M$, that is, by $\top$ if $M \geq 0$ and by $\bot$ otherwise. Invariants are then synthesized again from the strengthened guards.

Theorem 1.2 (Backward propagation)**.** *For every TBA $\mathcal{A}$ and every $n \in \mathbb{N}$, the automaton obtained by n propagation steps accepts the same timed words as $\mathcal{A}$.*

Proof. Strengthening guards can only remove runs. Conversely, consider an accepting run that takes $\ell \xrightarrow{g,Z} \ell'$ at event i and then a transition $\ell' \xrightarrow{g',Z'} \ell''$ at event $i+1$. The second guard holds at event $i+1$, so the values at that event are bounded by $Inv(\ell')$. Clock consistency gives $\mathsf{val}_\rho(i+1,x) = \Delta_i$ for $x \in Z$ and $\mathsf{val}_\rho(i+1,x) = \mathsf{val}_\rho(i,x) + \Delta_i \geq \mathsf{val}_\rho(i,x)$ otherwise; hence the strengthened guard $g \wedge Inv(\ell')[Z := 0]$ holds at event i. The argument uses clock consistency, so the theorem is stated on timed words. (Rocq: `propagate_accepts` and `propagate_n_accepts`.) □

Each round of the optimization pipeline of article Section 3.1 applies one propagation step. The tool repeats the rounds until the automaton is structurally equal to the previous one, as an OCaml value, or until 50 rounds have been performed; the bound guarantees termination, and the theorem holds for every number of steps, whether or not a fixed point is reached. On the 29 completed optimizer inputs, the greatest observed number of changed rounds is six (R6). The invariant computation at one location is summarized in Figure 1.

Example.. We illustrate the algorithm on the automaton of Fig. 2, which contains a two-location cycle $\ell_1 \to \ell_2 \to \ell_1$ and a self-loop at ℓ_2. It has one clock x, the initial location ℓ_0, and the transitions

$$\ell_0 \xrightarrow{a,\ Z=\{x\}} \ell_1, \quad \ell_1 \xrightarrow{b,\ x\leq 2,\ Z=\emptyset} \ell_2,$$
$$\ell_2 \xrightarrow{c,\ \top,\ Z=\emptyset} \ell_1, \quad \ell_2 \xrightarrow{d,\ x\leq 1,\ Z=\emptyset} \ell_2.$$

The edge $\ell_1 \xrightarrow{b} \ell_2$ requires $x \leq 2$, and the self-loop at ℓ_2 requires the tighter bound $x \leq 1$.

1. *Synthesis.* $Inv(\ell_1)(x) = ub_x(x \leq 2) = 2$. At ℓ_2, $Inv(\ell_2)(x) = \max(1, +\infty) = +\infty$, because the transition toward ℓ_1 does not bound x. At ℓ_0, the only outgoing guard is $\top$, so $Inv(\ell_0)(x) = +\infty$.

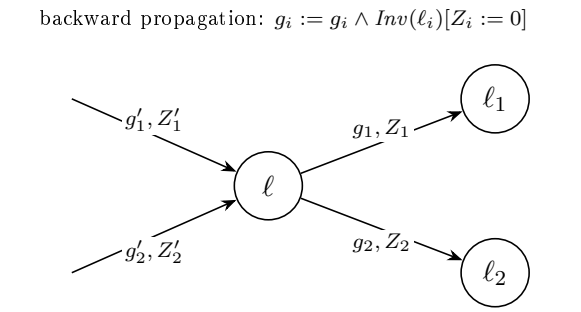


Figure 1: Invariant computation at one location. Successor invariants strengthen outgoing guards; upper bounds of the resulting guards resynthesize the source invariant. Reset sets determine which target-clock constraints become constant.

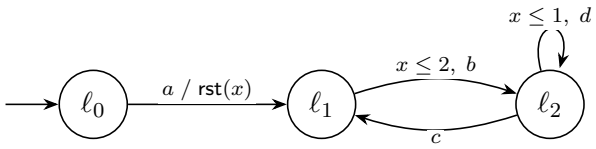


Figure 2: Automaton used to illustrate invariant propagation, with one clock x and actions a–d. Edges are labeled "guard, action / resets" (the guard is omitted when it is $\top$); only the edge $\ell_0 \to \ell_1$ resets x. The incoming arrow marks the initial location; acceptance plays no role here. The computed fixed point is $Inv(\ell_1) = Inv(\ell_2) = (x \le 2)$.

2. *Propagation.* The transition $\ell_2 \xrightarrow{c,\, Z=\emptyset} \ell_1$ becomes guarded by $\top \wedge Inv(\ell_1) = (x \le 2)$. The transition $\ell_0 \xrightarrow{a,\, Z=\{x\}} \ell_1$ receives $(0 \le 2) \equiv \top$ and is unchanged.

3. *Synthesis.* At ℓ_2, the outgoing guards are now $x \le 1$ and $x \le 2$, so $Inv(\ell_2)(x) = \max(1, 2) = 2$.

4. *Propagation.* The transition $\ell_1 \xrightarrow{b,\, x \le 2} \ell_2$ receives $x \le 2$, which it already contains, and the self-loop at ℓ_2 receives $x \le 2$, which is implied by $x \le 1$. Nothing changes.

The fixed point is $Inv(\ell_0) = \top$ and $Inv(\ell_1) = Inv(\ell_2) = (x \le 2)$. While the automaton remains in the cycle $\ell_1 \leftrightarrow \ell_2$, the clock x cannot exceed 2, and at every reachable valuation at least one outgoing transition of the current location is enabled. The invariants do not remove every blocking behavior:

since x is never reset in the cycle, time is blocked once $x = 2$. They make the upper bounds of the property automaton explicit, and Theorems 1.1 and 1.2 guarantee that they do not change its language.

1.3. Forward propagation of lower bounds and simplification

Entry bounds.. Dually, for a transition $t = \ell \xrightarrow{g,Z} \ell'$ and a clock x, the *entry bound* of x along t is 0 if $x \in Z$ and otherwise the largest bound $lb_x(g)$ of the constraints $x \geq m$, $x > m$, and $x = m$ of g, if any. The entry bound of x at a location ℓ' is the minimum of the entry bounds of x along the transitions entering ℓ'; for the initial location, the initial entry, with bound 0, also counts. If some entering transition gives no bound on x, the clock x has no entry bound at ℓ'. Since clocks are nonnegative and do not decrease while time elapses, every clock with entry bound m at ℓ' satisfies $x \geq m$ during every stay in ℓ'.

Forward propagation.. The forward step adds $x \geq m$ to the guard of every transition leaving ℓ', for every clock x with entry bound m at ℓ' (Rocq: `forward_accepts`). The entry bounds can also be read as lower-bound invariants: the two-sided automaton that carries the synthesized upper-bound invariants and the entry bounds as lower-bound invariants accepts the same timed words (`add_two_sided_invariants_accepts`). UPPAAL rejects lower-bound invariants, and their only role is to make infeasible transitions visible: after forward and backward propagation, a transition whose source location bounds a clock from below and whose target bounds it from above carries both bounds in its guard.

Simplification.. Three simplifications follow. First, a transition whose guard contains contradictory constraints is removed: a lower bound and an upper bound on the same clock with $u < l$, or with $u = l$ and one of them strict, or an upper bound $x \leq u$ with $u < 0$ or $x < u$ with $u \leq 0$. Such a transition is never enabled (Rocq: `remove_contradictory_accepts`). Second, the transitions leaving a non-initial location that has no entering transition are removed: such a location never occurs on a run after position 0 (Rocq: `remove_unreachable_accepts`). The iteration of the pipeline removes chains of such locations one step at a time. A location whose only entering transitions are self-loops keeps them, but it is unreachable from the initial location; the tool counts and prints only the locations reachable from the initial one and their transitions. Third, a transition whose action label is unsatisfiable, because it

requires two different actions or an action and its negation, is removed (Rocq: `remove_unsat_accepts`). Such labels arise when the translator receives an action a and the atom $\neg a$ of $\mathcal{T}(f)$ as independent propositions (the tool option `-indep-neg` of the translation article; its default negates the atom for a, which avoids most of them) or when a cube requires two different actions; removing them also enables clock merging, since the resets of a transition that can never be taken would otherwise prevent it.

1.4. Dead resets and merging of synchronously reset clocks

Dead resets.. A clock x is *live* at a location ℓ if some transition that tests x in its guard can be taken from ℓ, possibly after other transitions. The live locations are computed as a backward closure, along all transitions, from the sources of the transitions that test x; the other locations are *dead* for x. Since the correctness of the pass does not depend on how this closure is computed, the pass checks the two properties that the proof uses, namely that no transition leads from a dead location to a live one and that none tests x from a dead location, and then removes x from the reset sets of the transitions that enter a dead location. After such a transition, the value of x is never read again, so whether it is reset does not matter. The proof builds, from every accepted extended word, an extended word that differs only in the reset decisions of x, and conversely (Rocq: `remove_dead_resets_clock_accepts`). The pass is applied to every clock (`remove_dead_resets_accepts`).

Synchronous clocks.. The verified translation shares the clock of syntactically identical timed subformulas. Clocks of *different* subformulas can also be merged when they are reset synchronously. Two clocks $x \neq y$ are *mergeable* if

(i) every transition resets x if and only if it resets y;

(ii) no transition enters the initial location; and

(iii) every transition leaving the initial location resets x and does not test it.

Then, after the first event, x and y always have the same value, and every test of x can be replaced by the same test on y (Rocq: `merge_accepts`). The condition is checked by a Boolean function proved to imply it, and the pass

tries all pairs of reset clocks (`merge_all_accepts`). The merged clock x is no longer tested, so its remaining resets are removed by the dead-reset pass in the next round. This verified merging is restricted to synchronously reset clocks; merging clocks whose values coincide for other reasons is not covered.

1.5. Normalization of guards

Backward and forward propagation add constraints to guards. Every time a guard is strengthened, it is *tightened*: a constraint implied by another constraint of the same guard on the same clock is dropped, so that a guard keeps at most one lower bound and one upper bound per clock (an equality counting as both); for instance, $x \leq 5 \wedge x < 2$ becomes $x < 2$. The implication test compares bounds and strictness, and is proved sound (Rocq: `tighten_holds`). Normalization tightens every guard and also removes the lower bounds $x \geq m$ with $m \leq 0$ and $x > m$ with $m < 0$, which every clock value satisfies; it is needed after the merging of clocks, which renames constraints. The removal of trivial lower bounds relies on the nonnegativity of clock values, so the theorem is stated on timed words (Rocq: `normalize_accepts`).

1.6. Merging of transitions and of locations

Transitions.. Two transitions with the same source, target, and resets are merged when the union of their enabling conditions is again a conjunction. A transition is removed when another transition with the same source, target, and resets is *weaker*, that is, when its label and guard imply those of the other transition, constraint by constraint (subsumption). Two transitions whose labels, or guards, differ only by a literal and its negation, such as p and $\neg p$, $x < 2$ and $x \geq 2$, or $x \leq 5$ and $x > 5$, are replaced by one transition without that literal (resolution). The pass preserves the language because every enabled transition has an enabled counterpart with the same source and target, and conversely (Rocq: `merge_trans_accepts`). The other unions are disjunctive; they are formed by the export (article Section 5.2).

Locations.. A location ℓ' is merged into a location ℓ when both are accepting or both are not, and when (i) every transition leaving ℓ' is subsumed by a transition leaving ℓ, and (ii) every transition leaving ℓ is subsumed by one leaving ℓ'. A transition u subsumes t when it has a weaker label, the same resets, the same target once ℓ' is replaced by ℓ, and every constraint of its guard is implied by a constraint of the guard of t. In (i) the guard of t is

conjoined with the *entry invariant* of ℓ': the lower bounds $x \geq m$ that hold at every visit of ℓ'. They are the entry bounds of Section 1.3, except that a self-loop of ℓ' that does not reset x is ignored, because it preserves a lower bound (Rocq: `entry_lb_loop_sound`). Both orders of a pair are tried. The transitions entering ℓ' are redirected to ℓ, and those leaving ℓ' are removed. A run of the original automaton becomes a run of the merged one by replacing ℓ' by ℓ, using (i) and the invariant; conversely, a run of the merged automaton is followed in the original one, taking in ℓ' a transition that subsumes the one taken in ℓ, by (ii) (Rocq: `merge_states_all_accepts`; "state" in a Rocq name means location). The condition on the Büchi marking is necessary: merging an accepting location with a non-accepting one would accept runs that visit only the latter infinitely often.

2. General split construction and the two split policies

2.1. General split construction and preservation

Let $U_1, \ldots, U_r$ be the irredundant conjunctions of non-strict upper bounds of a disjunctive invariant; the construction makes one copy per U_k. At entry valuation v, the largest delay admitted by U_k is

$$d_k(v) = \min_{(x,b)\in U_k} (b - v(x)), \qquad d_k(v) = +\infty \text{ when } U_k = \varnothing.$$

The entry predicate for copy k is

$$G_k(v) \equiv U_k(v) \ \wedge \ \bigwedge_{\ell=1}^{r} d_\ell(v) \leq d_k(v).$$

Thus a copy is admitted only if its invariant holds at entry and no other copy permits a strictly longer stay. To encode the comparison without introducing a new clock, the implementation chooses an upper bound $y \leq b_y$ in U_ℓ and requires it to be reached no later than every bound $x \leq b_x$ in U_k. This gives $v(x) - v(y) \leq b_x - b_y$. The disjunction over possible minimizing bounds is represented in DNF. When some U_ℓ is empty, $d_\ell = +\infty$ and the comparison $d_\ell \leq d_k$ holds only for an empty U_k; the encoding then reduces to the constants true and false rather than to a choice of bound. An incoming edge reset set Z is handled by substituting zero for clocks in Z before conjoining G_k with the edge guard: a difference constraint with two reset clocks becomes a constant test, one reset clock gives a unary comparison, and no reset clock leaves a diagonal constraint.

The copy invariants are generated from the outgoing transition guards. If $\mathrm{ub}(c)$ extracts the non-strict unary upper bounds from a conjunctive guard clause c, the coverage contract is

$$\forall t \in \mathrm{Out}(q),\ \forall c \in \mathrm{DNF}(g_t),$$
$$\exists U \in \mathcal{U}(q) : \quad \mathrm{ub}(c) \Rightarrow U.$$

Here t has source q and guard g_t; implication means that every valuation satisfying all extracted upper bounds also satisfies every bound in U. The Rocq lemma `disj_inv_cover` proves this coverage for the deduplicated list $\mathcal{U}(q)$; it is an internal construction invariant, not a precondition imposed on callers. At the endpoint of any taken transition, the selected guard clause therefore entails a generated upper-bound conjunction. Since clocks only increase during the stay, those non-strict upper bounds also hold at each earlier valuation. This coverage argument is the key completeness condition for both the verified split and the all-copy comparison below.

All copies inherit the original location's accepting mark. No silent edge is added, and the proof relates runs by projecting each copy back to its original location. Thus repeated visits to an accepting location correspond to repeated visits to accepting copies, preserving Büchi acceptance. In Rocq, `split_accepts` proves timed-word preservation when the input invariant function is everywhere absent; `of_tba` establishes that precondition for the TBA export pipeline. The split invariant is absent or one conjunction per location by `split_inv_conjunctive`; the full export shape is established by `export_guards_conjunctive` and `export_invariants_conjunctive`. These are the internal representation theorems; the XML printer is not verified.

The optimized automaton has normalized conjunctive guards. Export first merges transitions that agree on source, target, event label, and resets, placing their guards into a disjunction. The location invariant is then represented by disjunctions of conjunctions of upper bounds. UPPAAL-oriented output requires each such invariant to be conjunctive because its location syntax admits a conjunction of clock conditions, so a disjunction must be encoded by a structural transformation [1]. Retaining a disjunction directly would not meet the target representation.

Every incoming edge is replicated to each regular target copy and conjoined with its reset-adjusted entry predicate; outgoing edges are replicated from every copy. This replication is essential: copying only the location and

assigning incoming edges to one arbitrarily chosen disjunct can truncate a stay and lose a timed word. The initial location uses a dedicated invariant-free copy because the first timed-word event has no incoming edge.

The representation cost follows directly from this construction. Let $K = \mathsf{kp}(D)$ be the number of source-copy slots used by the implementation and let $r(q)$ be the number of regular invariant copies of location q. Before guard explosion, each original edge t contributes $K\, r(tgt(t))$ split transition records. After guard explosion, the corresponding explicit-edge count is

$$E_{\text{split}} = \sum_{t \in T} K \sum_{k < r(tgt(t))} \left| \text{DNF}(g_t \wedge \text{subst}_{Z_t}(G_k)) \right| .$$

This count is taken before later normalization and pruning. It exposes two sources of growth: the number of invariant copies at the target and the DNF size of reset-adjusted entry guards. It is a structural count of the construction, not a complexity bound for the entire exporter.

Language preservation does not require selecting a copy with the longest admissible residence interval. We therefore mechanized a second policy, called the *all-copy split*: it retains the same conjunctive invariant copies, invariant-free initial copy, and accepting marks, but sends every incoming transition to every regular target copy with its original guard and reset set. It has no reset-adjusted entry predicate. The copy coverage property `disj_inv_cover` is established inside the Rocq construction, not assumed from an external parser.

Theorem 2.1 (All-copy split preservation)**.** *Let D be an input DTA with no pre-existing location invariants. The all-copy construction computes its upper-bound disjunctions and proves* `disj_inv_cover` *internally. Then the all-copy variant and D accept the same timed words.*

Proof. For soundness, project a run of the split automaton from each copy to its original location. Each copied transition keeps its original label, guard, and resets. The regular copies add their associated upper-bound conjuncts, but the input automaton has no location invariants, so projection forgets only this additional condition. The initial copy has no invariant, as required by the input contract. Accepting marks are inherited, so infinitely many accepting visits are preserved.

For completeness, fix an accepting run of D. At each noninitial location occurrence, consider the outgoing transition used by that run and a

DNF clause c of its guard that holds at the event ending the stay. By `disj_inv_cover`, some copy invariant U_k is implied by the upper-bound part of c. Hence U_k holds at that ending event. Each atom of U_k is a non-strict upper bound, so it also holds at every earlier valuation in the stay: clocks only increase during delay, and resets occur at its beginning. Thus copy k satisfies its invariant throughout this stay. The preceding transition is available to every regular target copy, and every outgoing transition is available from each source copy. Choosing copy k on entry therefore lifts this occurrence of the run. The initial occurrence uses the dedicated invariant-free copy. Making this choice at each occurrence preserves the event sequence and resets; inherited accepting marks give infinitely many accepting visits in the lifted run. The two inclusions establish language equality. □

Theorem 2.2 (Maximal-delay split preservation)**.** *Let D be an input DTA with no pre-existing location invariants. The maximal-delay variant (Rocq: `split_accepts`) and D accept the same timed words.*

Proof. Soundness is the projection argument of the previous proof: every copied transition keeps its label and resets, and its guard is the original guard conjoined with an entry predicate, so the projection of a run of the split automaton is a run of D. For completeness, fix an accepting run of D and a stay at a noninitial location q, entered with valuation v (after the resets of the incoming edge) and left after a delay Δ by a transition whose guard clause c holds at the end of the stay. By `disj_inv_cover` there is k^* with $\mathrm{ub}(c) \Rightarrow U_{k^*}$, so U_{k^*} holds at the end of the stay and $b - v(x) \geq \Delta$ for every $(x, b) \in U_{k^*}$, that is, $d_{k^*}(v) \geq \Delta$. Let k maximize $d_k(v)$ (`best_spec`; ties are all admissible, and $+\infty$ is allowed). Then $d_k(v) \geq d_{k^*}(v) \geq \Delta$, so every bound of U_k still holds at the end of the stay and, being non-strict upper bounds on increasing clocks, throughout it; and $G_k(v)$ holds by the choice of k, so the incoming edge, whose guard is conjoined with the reset-adjusted predicate G_k, can enter copy k. The outgoing transition is available from every copy. Making this choice at each occurrence lifts the run; accepting marks are inherited, and the initial occurrence uses the invariant-free initial copy. The two inclusions give language equality. □

The maximal-delay policy establishes a stronger local copy-choice property: at a given entry valuation, every admitted copy has an admissible residence duration at least as long as that of any other copy. The helper

Table 1: Measured split policies on direct synthetic TBAs. "Two-clock" is the `disjunctive_invariant_export` record of Table 5 and "six-branch" the disjunctive-invariant family at $k = 6$ of Table 8. Locations are graph-reachable locations. Times are means over 200 in-memory export calls.

Input	Policy	Reach. loc.	Edges	Guard atoms	Diagonal atoms	Time (ms)
Two-clock	Maximal delay	3	6	12	6	0.831
Two-clock	All-copy	3	10	8	0	0.712
Six-branch	Maximal delay	5	68	320	204	15.990
Six-branch	All-copy	5	68	64	0	28.731

lemma `best_spec` proves maximization, including $+\infty$ for an empty conjunction; ties remain admissible. This is neither global deadlock freedom nor a theorem about time divergence. Compared with all-copy replication, maximal-delay entry selection can introduce clock-difference constraints and more DNF clauses, while it may avoid entering copies that expire strictly earlier. The measured alternatives exhibit a trade-off (Table 1): on a two-clock example all-copy uses four more explicit edges but fewer guard atoms and no diagonal constraints; on a six-branch family both produce 68 edges, while all-copy uses fewer guard atoms and no diagonal constraints. All-copy is faster on the small case and slower on the six-branch case in these measurements. These are synthetic measurements, not a general performance ordering. The default remains the existing maximal-delay transformation; all-copy is separately verified and exposed for comparison.

3. Certificate-checked passes: conditions and proofs

3.1. Scheme and trusted base

Both passes have the same structure. An untrusted OCaml procedure (the *certifier*) returns a certificate; a boolean function written in Rocq and extracted tests local conditions on it; and a *try* function applies the transformation if the test returns `true` and returns its input otherwise. The preservation theorem is stated for an arbitrary certificate, in the form

$$\forall\, \mathsf{cert}\ \mathcal{A}, \quad \mathcal{L}(\mathsf{pass}(\mathsf{cert}, \mathcal{A})) = \mathcal{L}(\mathcal{A}),$$

where the pass is `prune_pass` or `alloc_pass` and `prune_certifier`, `certifier` are the function types of the certifiers. The proof never inspects how a certificate was found, so a wrong certifier can lower the quality of the result and cannot change its language. Nothing is added to the axioms

of the rest of the development. The proofs are in `MTL_to_TBA_DomPrune.v` (pruning and `split_cells`) and `MTL_to_TBA_ClockAlloc.v` (clock reuse), and the composition theorems are in `MTL_to_TBA_CheckedPasses.v`.

3.2. Removal of dominated transitions

Cells of letters.. Let $men(\mathcal{A})$ be the actions that occur in a label of $\mathcal{A}$ (`mentioned`). A *cell* is either $\mathsf{Some}\, a$ with $a \in men(\mathcal{A})$ or None, which stands for every action outside $men(\mathcal{A})$; `cells` lists them and `cell_of` maps an action to its cell. A label is a conjunction of literals over actions, and with one action at each position `label_at` decides it on a cell: a positive literal a holds on $\mathsf{Some}\, a$ only, and a negative literal $\neg a$ holds on every cell except $\mathsf{Some}\, a$. Lemma `label_at_iff` states that a label of $\mathcal{A}$ holds on an action p exactly when it holds on the cell of p. Dominance can therefore be tested on finitely many cells and not on all letters.

The certificate.. A certificate for $\mathcal{A}$ consists of a predicate *keep* on transitions, a relation *sim* on locations, and, for every location q, two lists of clocks $live(q)$ and $lle(q)$. The pruned automaton `prune_trans` has the same locations, initial location and accepting set as $\mathcal{A}$ and only the transitions that satisfy *keep*.

The checker.. `prune_ok` returns `true` iff $sim(q_0, q_0)$ for the initial location q_0 and, for all locations q_1, q_2 with $sim(q_1, q_2)$:

(P1) if q_2 is accepting then q_1 is accepting;

(P2) for every cell c and every transition t_2 out of q_2 whose label holds on c, there is a transition t_1 out of q_1 (`dom_step`) such that

- (a) $keep(t_1)$, and the label of t_1 holds on c;
- (b) the guard of t_2 implies the guard of t_1 (`guard_implied`), and every atom of the guard of t_1 has its clock in $live(q_1)$, or is an upper bound ($<$ or $\leq$) on a clock of $lle(q_1)$ (`guard_item_ok`);
- (c) (`flow_ok`) every clock $x \in live(tgt(t_1))$ is reset by both t_1 and t_2, or by neither and then $x \in live(q_1)$; every clock $x \in lle(tgt(t_1))$ is reset by t_1, or is reset by neither and $x \in lle(q_1) \cup live(q_1)$;
- (d) $sim(tgt(t_1), tgt(t_2))$.

The test is a finite conjunction over locations, cells and transitions. It is written with a lazy conjunction (`land`) because the extracted code of `andb` evaluates both arguments, and the check runs on automata with thousands of transitions; `dom_step_spec` and `prune_ok_spec` give the propositional reading above.

Meaning of live and lle.. For two runs, one of $\mathcal{A}$ at a location q_2 and one of the pruned automaton at a location q_1 with $sim(q_1, q_2)$, the clocks of $live(q_1)$ will have *equal* values and those of $lle(q_1)$ will have a value in the second run that is *less than or equal* to the value in the first. Equality is needed for guards of any shape; the order is enough for upper bounds, since the run of t_1 satisfies $x \leq c$ whenever the run of t_2 does and its value of x is not larger. The resets of the two transitions may differ on a clock that is dead after the step, which is what lets t_1 dominate a transition with a different reset set; this is the difference with syntactic subsumption (article Section 3), which requires the same target and the same resets.

The invariant.. Given a run π of $\mathcal{A}$ on an extended word ρ (positions $0, 1, \ldots$ with the actions, delays and clock values of ρ), the proof builds, by recursion on i, a run of the pruned automaton: `T2` i is the transition of $\mathcal{A}$ taken at position i; `T1` i q is, by `pick`, the transition that `prune_ok_spec` provides for q, the cell of the action at i and `T2` i; `qs` i is the location reached, with $\mathsf{qs}\,0 = q_0$; and the valuations `dpval` are those obtained by applying the resets of the chosen transitions to the delays of ρ. The record `dp_inv` states four facts at each position i:

$$sim(\mathsf{qs}_i, \pi_i) \;\wedge\; \mathsf{qs}_i < |\mathcal{A}|,$$
$$\forall x \in live(\mathsf{qs}_i),\ \mathsf{dpval}_i(x) = \rho_i(x), \qquad \forall x \in lle(\mathsf{qs}_i),\ \mathsf{dpval}_i(x) \leq \rho_i(x).$$

`dp_invariant` proves it by induction: the base case is the initial condition of the check, with both valuations equal to the initial one (so *live* and *lle* are respected at the initial location); the step uses `T1_spec`, then transfers the guard of t_2, which holds on ρ_i, to the guard of t_1 on `dpval` (`constraint_transfer` for clocks of *live*, `constraint_transfer_le` for upper bounds on clocks of *lle*), and finally restores the two clock facts after the delay and the resets by the conditions of `flow_ok`. Acceptance follows from (P1): the run of the pruned automaton is in an accepting location whenever the original run is, hence infinitely often (`prune_forward_run`, `prune_forward_ext`). The converse inclusion is immediate, since the pruned

automaton only removes transitions. `prune_trans_accepts` is the language equality under the hypothesis `prune_ok`, `try_prune_accepts` replaces it by the test, and `prune_pass_accepts` quantifies over the certifier.

Why each condition is there.. Dropping (P1) lets an accepting run be replaced by a rejecting one. Requiring equality on every clock, without the order on *lle*, would be sound but would forbid the dominance of a transition whose guard is an upper bound on a clock that the two runs have not kept equal; on CASAAL's automata for R2–R4 this restriction (`-prune-eq`, with splitting) removes no transition, and TChecker stores 890, 47,008 and 1,736,096 states as without pruning (`evaluation/baselines/baselines_posbadall_allcopy_prune_split_eq.tsv`). Using *lle* for a lower bound is unsound, because a smaller value can falsify $x \geq c$; hence `upper_cmp`. The condition on $tgt(t_1)$ in `flow_ok` propagates the two clock facts to the next position, and (d) propagates the simulation.

Splitting by cells.. `split_cells` replaces every transition t by one copy per cell c on which its label holds, with the label `cell_label` c: the single literal a for $\mathsf{Some}\, a$ and the conjunction of all literals $\neg m$, $m \in men(\mathcal{A})$, for None. The guard, resets and target are those of t. By `cell_label_holds` and `cell_label_inv`, the copy for c holds on an action exactly when the action is in the cell c and t holds on it, so the same words are accepted and the same runs exist (`split_cells_accepts`). Splitting is useful because the checker asks for a dominating transition for every cell of t_2 and removes t_2 as a whole.

Worked example.. Let the actions be a, b and the others. From a location q, three transitions go to the same location p with $sim(p, p)$, no resets and no clocks in play except $x \in live(q)$: t_3 with label a and guard $x \leq 5$, t_1 with label $\neg a$ and guard $x \leq 5$, and t_2 with label $\neg b$ and guard $x \leq 3$. On the cell $\mathsf{Some}\, a$, t_2 is dominated by t_3; on None, by t_1; but on $\mathsf{Some}\, b$ the label of t_2 does not hold, so no obligation arises. Hence `prune_ok` accepts a certificate that removes t_2. If instead t_1 had guard $x \leq 2$, t_2 would still be dominated on a but not on the other actions, and could not be removed as a whole; after `split_cells`, its copy on $\mathsf{Some}\, a$ is removable and its copy on None stays. Pruning alone removed only part of the extra states of CASAAL's automaton, and the splitting brought them to ours (article Section 9.2.4), which is consistent with labels that overlap without being nested.

The certifier.. The OCaml certifier (`tool/src/prune.ml`) computes *live* as the clocks read before being reset by lower bounds and equalities and *lle* as those read by any guard; takes *sim* as the greatest relation that satisfies (P1) and (P2) when every transition is kept; scans the transitions in order and drops one when another kept transition from the same location dominates it under *sim*; and recomputes *sim* on the kept transitions, dropping nothing if the recomputed relation no longer contains the initial pair. It is a heuristic, and its result is validated by `prune_ok`.

The equality-only baseline.. The option `-prune-eq` changes only the certifier: it sets $\mathit{live}(q)$ to the set that is otherwise $\mathit{lle}(q)$, the clocks read by any guard (lower bound, equality or upper bound), so that every clock read after q must be equal in the two runs, and *lle* is that same set. The greatest relation *sim* and the greedy choice of the transitions to drop are recomputed by the same code with these sets, and the checker `prune_ok` is the unchanged verified function; it accepts these certificates too. The two configurations therefore differ only in the clock relation that the certificate is allowed to claim.

A dominance witness that needs the order.. Take R1, the bounded response $\Box(a_1 \to \Diamond_{\leq 5} b_1)$, as printed by CASAAL and imported. Location q_0 (no pending obligation) has a transition to q_1 (pending) on every letter that resets x, and loops on b_1 and on $\neg a_1$; q_1 has loops and a transition back to q_0 under the guard $x \leq 5$. On the cell $\mathsf{Some}\, a_1$, the transition $t_2 = q_1 \to q_1$ (guard $x \leq 5$, no reset) can only be matched from q_0 by $t_1 = q_0 \to q_1$, which resets x. For q_0 to simulate q_1, which is what lets the checker drop the copy of $q_0 \to q_1$ on the letters b_1 and $\neg a_1$ that $q_0 \to q_0$ already covers, `flow_ok` must hold for (t_1, t_2) with target q_1. There x is read only by the upper bound $x \leq 5$, so with the full certifier $x \in \mathit{lle}(q_1)$ and $x \notin \mathit{live}(q_1)$, and the second part of `flow_ok` holds because t_1 resets x: the run of t_1 has $x = 0$, which is at most the older value of x in the run of t_2, and $x \leq 5$ still follows. In the equality-only certifier $x \in \mathit{live}(q_1)$, and the first part of `flow_ok` asks that t_1 and t_2 both reset x or neither does; they differ, so q_0 does not simulate q_1 and nothing is dropped (`evaluation/prune_witness.sh` prints this with `PRUNE_DEBUG`, and its output is in `evaluation/prune_witness_R1.txt`: it shows $\mathit{live}(q_1)$ empty and $\mathit{lle}(q_1) = \{x\}$ in the first mode, and $\mathit{live}(q_1) = \{x\}$ in the second, with the pair (q_0, q_1) kept in the first and lost in the second). After splitting by cells and pruning, R1 has 7 transitions with the full certifier and 10 with the equality-only one. R2–R5 combine n such pending obligations, and the equality-only certifier removes nothing there (article Section 9.3).

3.3. Reuse of clocks

The certificate and the renamed automaton.. The certificate is an allocation α (a function on clocks) and, for every location q, a list $\mathit{live}(q)$. `allocate_clocks` replaces every clock x of a guard atom by $\alpha(x)$ (`alloc_item`) and the reset list of every transition by `nodup(map` α *resets*`)` (`alloc_resets`), and leaves locations, labels, invariants, initial location and acceptance unchanged.

The checker.. `alloc_ok` tests, for every transition t from q to q', and for the initial location q_0:

(G) every clock of the guard of t is in $\mathit{live}(q)$;

(F) every $x \in \mathit{live}(q')$ is reset by t or belongs to $\mathit{live}(q)$;

(R) for every r reset by t and every $y \in \mathit{live}(q')$, $\alpha(r) = \alpha(y)$ implies $r = y$ (`same_class_eq`);

(I) for $x, y \in \mathit{live}(q_0)$, $\alpha(x) = \alpha(y)$ implies $x = y$.

`alloc_ok_sound` shows that a successful test yields the record `alloc_valid`, which has the four conditions in propositional form. No other property of α is needed: α need not be idempotent, and the sets *live* may be any over-approximation that satisfies (G) and (F), so the certifier is free to be conservative.

Forward direction.. Given a run of $\mathcal{A}$ on ρ, the word `fwd_word` gives to the clock c of the renamed automaton the value of the clock of $\mathcal{A}$ that was last reset among the class of c (`fwd_reset`, `init_val`); at the initial position α is injective on $\mathit{live}(q_0)$ by (I), so the initial values of the live clocks can be placed in their classes. `fwd_invariant` states that at every position every clock of *live* of the current location has in the renamed word the value that it has in ρ. It is proved by induction: (F) says that a live clock after the step was reset (then its value is the delay) or live before (then, by induction, correct), and (R) says that resetting a class does not disturb a live clock of the same class unless it is the reset clock itself. With (G), `item_transfer` gives the guard of the renamed transition from the guard of the original.

Backward direction.. Given a run of the renamed automaton, `bwd_word` gives to each clock of $\mathcal{A}$ the value of its class and `backward_ext` builds the run of $\mathcal{A}$ by choosing, at every position, the first transition of $\mathcal{A}$ that maps to the transition taken (`pick_spec`), so no choice axiom is used. `item_`

`transfer_back` gives the guard of that transition from the renamed guard, using (G) so that the clocks that it reads are live and correct. Locations, labels and acceptance are the same, hence the runs have the same accepting positions. `allocate_clocks_accepts`, `try_allocate_accepts` and `alloc_pass_accepts` are then the language equalities under the hypothesis, under the test and for every certifier.

The certifier.. The OCaml certifier (`tool/src/alloc.ml`) computes the least *live* sets satisfying (G) and (F) by a fixpoint, builds the interference graph (an edge between a reset clock and each clock live at the target, and between the clocks that are live at the initial location), and colours it greedily with DSATUR; each clock goes to the first clock of its colour. It is heuristic: DSATUR does not find the fewest colours in general, and the liveness sets are over-approximations. The fuzz of the article (552 random allocations) exercises the checker, not the proof.

3.4. Composition

`MTL_to_TBA_CheckedPasses.v` states, for every pruning certifier, every allocation certifier, every n and every well-formed f, that a timed word satisfies f iff the export of the result of optimize, then split_cells, prune_pass and alloc_pass applied to compile(f) accepts it (`MTL_to_exported_checked_correct`); the variants `MTL_to_exported_all_checked_correct` and `MTL_to_exported_checked_correct_with` cover the all-copy export and the Spot-parametrized chain, with the same hypothesis on the propositional automaton as the article's Theorem 7.1; `MTL_to_exported_checked_correct_recur_weak_with` is the statement for the chain of the tool (the translation of recur(f) with the weakened encoding as Spot contract). The zero-initialized versions `MTL_to_exported_checked_correct0_with` and `MTL_to_exported_checked_correct0_recur_weak_with` add the hypothesis that `init_free` returns true on the final exported automaton, after pruning and clock reuse, through the lemma `checked_export_accepts0`; they conclude acceptance with all clocks initialized to 0 and hold for every pair of certificate producers. The two certificate-checked developments add 1,535 lines and 50 lemmas and theorems to the project; `Print Assumptions` reports only the standard library principles listed in the article.

3.5. Evidence on the CASAAL workload

The article's pruning result is on CASAAL's automaton for the positive form of the bounded responses, read by the importer of article Section 9.2.1.

Table 2: CASAAL's automaton for the positive form of Rn, from the DOT file (acceptance marks printed: none). Letter-level transitions are the (source, target, action, guard, resets) tuples that the printed labels denote when the actions are mutually exclusive; "covered" counts disjuncts all of whose tuples occur in another disjunct of the same printed edge.

n	locations	edges	disjuncts	letter transitions	distinct	covered
2	4	16	25	51	45	6
3	8	64	125	216	189	27
4	16	256	625	837	729	108
5	32	1,024	3,125	3,078	2,673	405

Two facts show that the importer's acceptance conversion is not what makes this automaton large. First, for R2–R5 CASAAL's output carries no acceptance mark, so the importer keeps the printed locations (2^n) and marks every state accepting, as the label of no state contains an unbounded eventuality; no location is split and no transition is dropped. Second, the redundancy is present in the printed labels. Table 2 counts, from CASAAL's own DOT files (`evaluation/casaal_label_redundancy.py`), the printed edges, the disjuncts of their labels, the transitions between letters that these denote (with mutually exclusive actions, as in the article) and the disjuncts that are covered by the other disjuncts of the same printed edge. The transitions between letters, 51, 216, 837 and 3,078, are the explicit edges that TChecker receives from CASAAL's automaton (article Section 9.2.4); 12 to 13% of them repeat a transition of the same printed edge, and the others differ in target, guard or resets, which is the overlap that the checked pruning removes after splitting. What the importer chooses is the reading of an event literal as one letter (mutually exclusive events) and the expansion of a label into one transition per disjunct, which TChecker's input format requires.

3.6. Clock reuse on natural benchmark automata

The article reports the timing of clock reuse on constructed families. Table 3 adds the benchmark automata whose negated form loses a clock and whose product with the positive form (the reference workload) TChecker completes, measured in the same way (five interleaved runs, one core, `evaluation/alloc_timing.tsv` and `alloc_timing_natural.tsv`). The product for R4 was not run, as it needs more than the limit of 300 s per run for ten runs.

Table 3: Clock reuse on the benchmark automata whose negated form loses a clock, product with the positive form, all-copy export, TChecker `couvscc`. Stored states and verdict are identical with and without reuse; the ratio is the median time with reuse over the median time without.

Req.	clocks	states	median (s)	median, reuse (s)	ratio
F11	2, 1	67	0.0006	0.0005	0.90
F12	2, 1	1,688	0.0185	0.0164	0.89
F13	2, 1	87	0.0008	0.0007	0.81
R2	2, 1	7,513	0.1523	0.1348	0.89
R3	3, 1	168,484	9.1669	7.4508	0.81

4. Tool details

4.1. Verified speedups of the implementation

The definitions of the passes were first written for the proofs, with list scans that are linear in the transitions for every location or transition they inspect. On the larger automata of the evaluation these scans, not the passes themselves, dominated the time of the pipeline. We replaced them without changing what the passes compute, and each replacement carries a proof of equality with the definition it replaces, so that every theorem of the paper applies to the faster code that is extracted.

- *Per-location tables* (`forward`, `propagate`, `split`): the entry guards, the incoming and outgoing transitions and the invariant information of a location are computed once per location into a list indexed by the location, with a fallback to the original function for out-of-range locations (lemmas `forward_fast_eq`, `propagate_fast_eq` and `split_fast_eq`; the last compares closures and uses functional extensionality, which is already among the axioms of the development).

- *Location merging*: the test that two locations can be merged reads the transitions leaving the two locations instead of the whole relation, and evaluates the cheaper conjunct first (`states_mergeable_fast_eq`).

- *Initial-value check*: the search for the locations from which a clock is read before it is reset keeps a set without duplicates and scans only the transitions entering the locations found in the previous step. No equality lemma is needed: the proof of `init_free` uses only the checker that validates whatever set the search returns.

- *Reset completion*: the list of clocks of the formula, a walk over all of its timed subformulas, was recomputed for every transition; it is now computed once (`compile_with_fast_eq`, by computation).

Two further changes concern the call to Spot and are not covered by the proofs; they sit outside the trust boundary already drawn around Spot. Spot's simulation-based reduction is switched off at level `any` (`-x ba-simul=0`), which cuts Spot's time on the nested requirements by a factor of up to 2.8 without changing the automaton it returns on our benchmark, and the tool reads Spot's HOA output, whose guards are Boolean formulas, instead of LBTT. The two readers give the same automata: the messages of the tool, which report the sizes after every stage, are identical on 184 runs (the benchmark patterns other than N6–N8, R7 and R8, both polarities, both levels, with one and with the default number of rounds); the XML and DOT files are identical on 144 of them and differ on the others only in the order of the conjuncts of a guard or of the labels of a transition. The speedups added no axiom, and the generated automata are byte-identical to those of the previous implementation (UPPAAL XML, DOT and the messages of the tool) on 92 runs: the benchmark patterns other than N6–N8, R7 and R8 (and, for the positive form, other than the largest ones), in both polarities, with one optimization round and with the default number of rounds. The effect is given in article Section 8.3. The development was later reworked so that its only axioms are `ClassicalDedekindReals.sig_forall_dec` (from the construction of the real numbers) and functional extensionality, with decision procedures replacing excluded middle, definite description and choice; the byte-identity comparison above predates this change and was not repeated.

4.2. Input contract for an external TBA

The interface starts from a finite TBA value (L, ℓ_0, X, E, Acc) in the Rocq representation. Its clock set X is `Clockroot`, the timed subformulas of a formula `root` fixed by the development; an automaton with k clocks is represented by choosing any formula with at least k distinct timed subformulas, which then serve only as clock names. Its transitions carry event conditions, finite conjunctions of unary clock comparisons, and reset sets; the starting value has no location-invariant annotation. The invariant synthesis pass derives that annotation from outgoing guards. The theorem does not require a complete or deterministic transition relation, a particular number of clocks,

or a connection to a temporal-logic translator. A third-party producer can therefore use the proved functions after converting its automaton to this representation and naming its clocks in this way.

The conversion is a semantic boundary: it must retain the alphabet and the meaning of each event condition, preserve which clocks are tested and reset, and map accepting locations without changing their marking. The paper's repository provides the Rocq type and extracted optimizer, but not a verified reader for arbitrary third-party formats. Any such reader belongs to the trusted interface until its correspondence with the Rocq value is proved. The same applies to interpreting the result in a target checker: export proves language preservation for the Rocq representation, while a separate bridge is needed to identify the checker's parsed model with that value. This boundary keeps the central optimization theorem independent of the producer while making the work needed to reuse it explicit.

4.3. Build and artifact layout

The verified functions are defined under `proof/` and extracted into `tool/src/optim.ml`. The project Makefile compiles the Rocq development and extracts the executable functions. Hand-written components include the parser, Spot invocation, the reader of Spot's HOA (or LBTT) output, iteration driver, and XML/DOT printers; the script that removes unreachable extracted definitions and the Rocq/OCaml toolchains are also part of the implementation's trusted base. The repository includes the machine record, raw benchmark runs, TSV summaries, table generator, formulas, UPPAAL models and queries, and reproduction instructions in `README.md`.

Table 4 separates the verified transformations from the interfaces that connect them to the surrounding tools.

In the extracted code, structural equality decides formula and clock identity, and Boolean checks implement conditions whose soundness is proved in Rocq. Real comparisons are realized by OCaml floating-point operations. Input constants lie in $[0, 2^{30}]$; the inspected operations produce only such constants, their differences, or their negations. These values are exactly representable by the 53-bit significand of a double, and the printed constants fit UPPAAL's 32-bit integer domain. This arithmetic argument is based on inspection of the extracted implementation rather than a theorem about OCaml or the printer. Rational constants can be handled by a common time scaling only when every scaled bound remains an integer in $[0, 2^{30}]$; timing constants in any surrounding system model must be scaled by the same

Table 4: Runtime stages and their proof boundary. The preservation theorems begin with a TBA in the Rocq representation; the Spot-based formula front end is one client of the verified optimizer and exporter.

Stage	Implementation	Guarantee or boundary
TBA input and adapter	Rocq TBA value or a parser/converter supplied by the caller	Pass results start at this representation; an external adapter is outside the theorem unless separately verified.
Spot-based front end	Spot call and hand-written HOA/LBTT reader in `mtl2tba`	Used by the integrated formula workflow; outside the TBA preservation theorems.
Optimization rounds	Extracted from `MTL_to_TBA_Optimizations.v`	`optimize_accepts`, for every TBA and every round count.
Iteration driver	Hand-written OCaml structural comparison; stops at equality or 50 rounds	The proof does not assume convergence or a particular stopping point.
UPPAAL export	Extracted from `MTL_to_TBA_Export.v`	`export_accepts` and conjunctive guard/invariant results.
Symbolic grouping	Extracted from `MTL_to_TBA_Symbolic.v`	`symbolic_ext_accepts`.
XML, DOT, and PDF output	Hand-written printers and Graphviz	Outside the Rocq language-preservation results.
Numeric realization	OCaml floating point with integer bounds in $[0, 2^{30}]$	Exactness follows from inspection of the extracted operations, not a mechanized theorem.

factor.

5. Translator-independent synthetic evaluation

5.1. Translator-independent synthetic TBA suite and pass ablation

To exercise the interface without the MTL translator, we construct 16 small synthetic automata directly as Rocq-representation TBA records in `evaluation/synthetic_tba_bench.ml`. Their clocks are named by the six timed subformulas of a fixed formula, the conjunction of six hatted primitive formulas over three events; this formula is never translated and serves only as a supply of six clock names. Locations, event labels, unary guards, resets, and accepting locations are written by hand for each record. The suite is explicitly synthetic: it does not parse a third-party format and is

Table 5: Sixteen translator-independent synthetic TBA records. Count tuples are declared locations / transitions / distinct clocks / reset occurrences / unary guard atoms; the export tuple adds diagonal guard atoms. The I field gives nonempty invariant locations:bound atoms. Optimization and export times are per-call means over 200 repeated in-memory runs; RSS is the peak for a separate single-model process. Full per-pass and reachability data are supplied in the artifact.

Model	Input	Optimized	Rounds	Opt. (ms)	RSS (KiB)	Export	Exp. (ms)
stable	1/1/0/0/0	1/1/0/0/0; $I = 0:0$	0	0.008	6364	2/1/0/0/0/0; $I = 1:0$	0.089
redundant_guard	2/2/1/1/3	2/2/1/1/2; $I = 2:2$	1	0.133	6460	4/2/1/1/2/0; $I = 2:2$	0.197
contradictory_guard	2/2/1/0/2	2/1/0/0/0; $I = 0:0$	1	0.042	6428	4/1/0/0/0/0; $I = 1:0$	0.073
unsatisfiable_label	2/2/0/0/0	2/1/0/0/0; $I = 0:0$	1	0.026	6352	4/1/0/0/0/0; $I = 1:0$	0.143
isolated_source	3/2/0/0/0	3/1/0/0/0; $I = 0:0$	1	0.027	6340	6/1/0/0/0/0; $I = 1:0$	0.264
dead_reset	2/2/1/2/0	2/2/0/0/0; $I = 0:0$	1	0.052	6468	4/2/0/0/0/0; $I = 2:0$	0.150
normalization_dead_reset	2/2/1/1/1	2/2/0/0/0; $I = 0:0$	1	0.110	6336	4/2/0/0/0/0; $I = 2:0$	0.491
synchronous_clock_merge	2/3/2/2/2	2/3/1/1/2; $I = 1:1$	2	0.673	6300	4/3/1/1/2/0; $I = 2:1$	1.279
duplicate_transition	2/3/1/2/3	2/2/1/1/2; $I = 2:2$	1	0.267	6364	4/2/1/1/2/0; $I = 2:2$	0.478
equivalent_locations	3/4/0/0/0	3/2/0/0/0; $I = 0:0$	2	0.104	6404	6/2/0/0/0/0; $I = 2:0$	0.220
disjunctive_invariant_export	3/3/2/0/2	3/3/2/0/2; $I = 0:0$	0	0.095	6488	9/6/2/0/12/6; $I = 3:2$	1.154
backward_tightening	2/2/1/0/2	2/2/1/0/2; $I = 2:2$	1	0.093	6360	4/2/1/0/2/0; $I = 2:2$	0.204
forward_lower_bound	3/3/1/0/2	3/3/1/0/4; $I = 2:2$	1	0.141	6460	6/3/1/0/4/0; $I = 3:2$	0.654
forward_contradiction_cascade	6/7/1/0/3	6/4/1/0/4; $I = 1:1$	2	0.437	6428	12/4/1/0/4/0; $I = 3:1$	1.082
backward_contradiction_cascade	5/5/1/0/4	5/1/0/0/0; $I = 0:0$	2	0.201	6320	10/1/0/0/0/0; $I = 1:0$	0.180
clock_merge_contradiction_cascade	5/5/2/6/4	5/3/1/2/1; $I = 1:1$	2	0.622	6484	10/2/1/2/1/0; $I = 2:1$	0.598

not evidence of industrial-model performance. Its records cover redundant guards, contradictory guards, unsatisfiable labels, a source with no incoming edge, dead resets, synchronously reset clocks, duplicate transitions, equivalent locations, a disjunctive-invariant export, a stable no-change input, and targeted backward/forward propagation and pass-interaction cases.

The harness reports counts after every pass in every round and checks that its profiled pass sequence is structurally equal to the extracted `optimize_step` on each round. It also times the unprofiled iterative optimizer and export as means over 200 repeated in-memory calls per model. Table 5 lists the counts and times; standalone process RSS ranges from 6,300 to 6,488 KiB. These short timings and small, purpose-built records are microbenchmark evidence only; the raw pass table and per-model RSS file are supplied for reproduction. Per-pass times are recorded but are too small to support fine-grained speed rankings.

The targeted examples exercise both bound-propagation directions. Backward synthesis tightens an incoming $x \leq 5$ guard to $x \leq 2$. Forward propagation adds an entry lower bound $x \geq 2$ to outgoing edges. In the forward-cascade input, an alternate $x \leq 10$ branch prevents backward synthesis from creating the contradiction; forward propagation makes the $x \leq 1$ branch contradictory, and the following pruning stages remove its unentered chain. Omitting forward propagation retains all seven input edges, while the full optimizer leaves four. A separate backward cascade isolates that pass.

Table 6: Measured trace for the six-clock synchronous-reset synthetic input. Counts are declared / reachable locations, transitions, active clocks, reset occurrences, guard atoms, invariant locations / atoms, and diagonal guard atoms. The export row is the internal explicit DTA.

Stage	L/R	E	C	Resets	Guard atoms	Inv. L/A	Diag.
Input	2/2	7	6	6	6	0/0	0
R1: unsatisfiable-label removal	2/2	5	6	6	4	0/0	0
R1: dead-reset elimination	2/2	5	4	4	4	0/0	0
R1: synchronous-clock pass	2/2	5	4	4	4	1/1	0
R2: dead-reset elimination	2/2	5	1	1	4	1/1	0
Optimized fixed point	2/2	5	1	1	4	1/1	0
Explicit export	4/2	5	1	1	4	2/1	0

Table 27 shows input/output counts and leave-one-operation-out effects.

The normalization/reset example removes a tautological $x \geq 0$ guard before dead-reset analysis, allowing its reset and active clock to be removed. The synchronous-clock example first merges the clocks, then a later round detects the conflicting $x \geq 5 \wedge x \leq 3$ guard and prunes the branch. Other records measure unsatisfiable-label elimination, no-incoming-source pruning, transition subsumption, and equivalent-location merging. The no-incoming operation is a local predecessor test, not full graph reachability. These interactions explain the iterative design; preservation for each finite iteration count follows from the proved pass composition.

Table 6 gives the measured stage interaction for the largest synchronously reset synthetic input. Unsatisfiable-label removal first reduces the relation from seven edges to five. Dead-reset elimination then lowers the active-clock count from six to four. The synchronous-clock pass changes the synthesized invariant, and the next round's dead-reset pass removes the remaining redundant resets, leaving one active clock. This is a measurement of the extracted implementation; the language-preservation claim follows separately from the proved pass composition.

The synthetic disjunctive-invariant case is unchanged by optimization, then grows from three locations and three transitions to nine declared locations (three of them reachable) and six explicit transitions after export. The output has conjunctive invariants and six diagonal guard atoms that select maximal-delay copies. This is a measured size cost of export; the output-shape guarantee itself follows from `export_guards_conjunctive` and `export_invariants_conjunctive`.

5.2. Pass ablation and parameterized synthetic families

To measure pass interactions, we repeated the same composite pipeline while omitting one operation at a time from sixteen fixed synthetic input records. Each modified composite was iterated to structural stability (at most 50 changed rounds); this is a measurement driver, not a separate Rocq theorem about the ablated variants. The full variant was checked for structural equality with the extracted optimizer on each input. Table 7 reports aggregate reachable locations, total transition records, active clocks, changed rounds, and median repeated-call time. The raw TSV gives each record and its counts under every pass.

The aggregate ablation shows effects that selected records were designed to expose. Relative to the full variant (33 transitions and nine active clocks summed across the suite), omitting backward synthesis leaves 34 transitions and ten clocks; omitting forward propagation leaves 36 transitions; and omitting contradiction removal leaves 43 transitions. Omitting pruning of sources without incoming edges leaves 39 transitions. Normalization and dead-reset omission raise the aggregate active-clock count to 12 and 13, respectively. Omitting synchronous-clock merging leaves 35 edges and 11 clocks; omitting transition merging leaves 35 edges, while omitting equivalent-location merging leaves one more reachable location. These are descriptive differences on a deliberately selected synthetic suite, not estimates of typical pass effects. Per-call medians are too small for fine-grained pass runtime rankings; process RSS includes runtime overhead.

We also constructed five size-parameterized families directly in the formal TBA representation, without a third-party parser. Table 8 reports the sizes shown there; the parameter k is the number of parallel edges (duplicate edges), the chain length (unreachable cascade), the number of bounds (redundant bounds), the number of clocks (synchronous resets) and the number of branches (disjunctive invariant). In the duplicate-edge family, inputs grow from two to 33 edges while optimization reduces each nontrivial instance to two. The unreachable-cascade sweep covers sizes 1–50, 51, and 64 (the table shows selected sizes); rows at the 50-round driver cap are marked with a dagger and report capped, not necessarily stable, outputs. A separate probe confirms that size 51 still changes on round 51 (two edges remain after round 50), while size 64 still changes on rounds 51 and 52. The size-50 chain has one edge after round 50 and is unchanged by round 51, but the driver conservatively warns because it did not perform that stability check. Synchronously reset families with two to six clocks reduce to one active clock

Table 7: Iterated leave-one-operation-out ablations on sixteen fixed synthetic TBA records. Reachable-location, transition, and active-clock counts are summed across the same inputs; rounds and runtime are medians across inputs. Runtime is the median of each model's mean over 200 in-memory runs. Each variant repeats its retained composite to structural stability. The full variant is checked against the extracted optimizer; the modified ablation driver is not a separate Rocq theorem.

Variant	$\sum L_{\text{reach}}$	$\sum E$	$\sum C$	med. rounds	med. time (ms)	cases with E above full
Full optimizer	30	33	9	1	0.102	0
Omit bound synthesis/backward propagation	31	34	10	1	0.104	1
Omit forward propagation	33	36	9	1	0.085	1
Omit contradictory-edge removal	40	43	11	1	0.102	4
Omit unsatisfiable-label removal	31	34	9	1	0.105	1
Omit no-incoming-source pruning	30	39	10	1	0.098	4
Omit guard normalization	30	33	12	1	0.107	0
Omit dead-reset removal	30	33	13	1	0.101	0
Omit synchronous-clock merging	32	35	11	1	0.091	1
Omit transition merging	30	35	9	1	0.084	2
Omit equivalent-location merging	31	35	9	1	0.068	1

in two rounds. The disjunctive-invariant export grows from 10 edges and 10 diagonal guard occurrences at $k = 2$ to 68 edges and 204 diagonal occurrences at $k = 6$. These counts expose representation costs in a deliberately constructed stress family; they are not industrial-model measurements or a complexity bound. For every finite number of rounds, including a capped result, `optimize_accepts` preserves language. Reaching the iteration cap does not justify calling that result a fixed point. Per-size intermediate records and the exact cap probe are preserved in the dated run artifacts.

5.3. Optimizer and export on independently generated TBAs

This subsection documents the "Independently generated TBAs" experiment of the main text (the scripts and per-input results are in `evaluation/independent/`). Neither generator uses the MTL translator.

Generation. `graph`: 300 random timed Büchi graphs with 6 to 12 locations, 2 to 4 clocks, 3 or 4 events and constants up to 8. `monitor`: 300 specification-style monitors (response chains with timeouts, periodic events, watchdogs, rate limiters, windows), each with optional clocks reset in lockstep, dead resets, unreachable branches and duplicated or subsumed edges. An input is kept when between 1% and 95% of the test words are accepted, so that neither the empty nor the universal language is tested.

Procedure. The unmutated extracted optimizer and exporter (`tba_opt`) run on each input with the maximal-delay and with the all-copy policy. The

optimized TBA and the exported XML are compared with the input by the bounded word test of the main text: 20,000 ultimately periodic words, six initial clock valuations per word (the first all zero) shared by both automata, and 60 loop iterations after the warm-up (the default of 12 produced one spurious difference caused by the horizon, see the main text). Acceptance is approximated on the unrolled word, so the test can miss a difference. Table 9 gives the results per family.

Table 9: Optimizer and export on 600 independently generated TBAs. Transitions are summed over the inputs; the optimized counts are those of the maximal-delay run (the optimizer is the same for both policies). Runs are all completed runs (two per input); Mism. counts inputs on which the optimized TBA or an exported XML differs from the input on some tested word.

Family	Inputs	Runs	Trans. in	Trans. out	Reduced	Mism.	`init_free` fails
graph	300	600	5,070	3,586	285	0	279
monitor	300	600	2,492	1,930	249	0	144
total	600	1,200	7,562	5,516	534	0	423

The number of locations is unchanged by the optimizer on these inputs (2,421 and 1,130). The `init_free` column counts inputs for which the check fails; the bounded word test then compares the languages from the sampled initial valuations, which include the all-zero one, but the zero-initialized guarantee of the theorems does not cover these inputs. The inputs are our own, and the test is bounded; it is evidence for the extracted code on these inputs, not a proof.

6. Generated-group optimizer results

6.1. Optimization effects on the generated group

In R6, Spot emits 64 states and 15,625 cube-split transitions. The recorded post-reset-completion optimizer input also has 64 locations and 15,625 transitions; the equality of these two edge counts is specific to R6, and the Comp. column is the authoritative optimizer boundary. Six optimization rounds reduce the transition relation to 640 edges and retain six clocks. Export then produces 1,837 explicit edges, 63 invariant-bearing locations, and 3,360 clock-difference guard occurrences; symbolic grouping represents the output with 448 transitions. The larger explicit edge count results from splitting disjunctive guards and adding entry conditions for

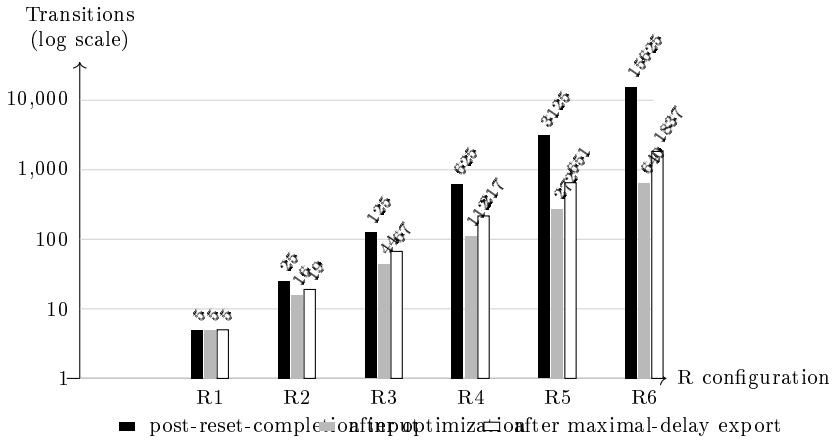


Figure 3: Post-reset-completion input, optimized TBA, and maximal-delay explicit export for R1–R6. The logarithmic vertical axis makes the R6 concentration visible. These are positive forms measured with Spot level `small` and the implementation before the speedups (R7 and R8 did not complete); the counts are five-run medians, and Comp. (the post-reset-completion input) is the optimizer boundary, regardless of whether an upstream Spot count happens to match it.

invariant copies. It measures the cost of the target representation, not a reversal of the optimizer's reduction. In the fresh run, median optimizer and export times are 17.807 s and 10.666 s, respectively; these are times of the build before the verified speedups of article Section 8.3, with which the optimization of the positive form of R6 takes 0.29 s. Figure 3 shows the concentration on R6. The generated-group table separately reports the stage after reset completion so that upstream processing is not folded into these optimizer measurements.

The same distinction matters for the other generated inputs. For example, F11 starts from its actual four-location, seven-transition, two-clock TBA after reset completion; two rounds leave three locations and four transitions and remove one clock. Two locations have the same loop and the same exit on p to the accepting location, except that the exit of the first also carries $x \geq 2$; the second is entered only when $x \geq 2$ holds and x is not reset afterwards. They are merged because the guards are compared under that entry invariant, which equality of guards cannot do. N8 spends most of its end-to-end time in Spot, whereas its recorded optimizer and exporter stages take 0.33 s and 0.70 s. These cases show that the optimizer's cost depends on its input relation and that the upstream producer can dominate total tool time. They do not

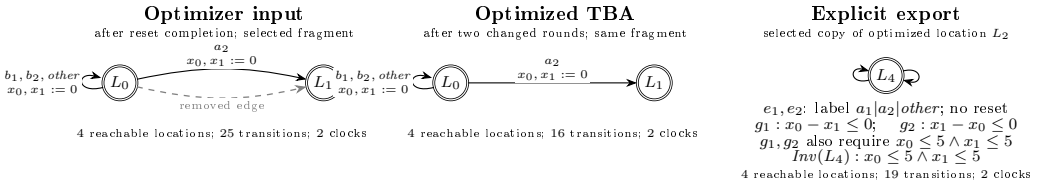


Figure 4: Selected subgraphs from the actual generated R2 files. The dashed input edge is absent after optimization; only one of the nine removed transitions is shown. The first two panels show a retained edge and loop. The export panel follows the optimized L_2 loop to its L_4 copy, where one source loop becomes two conjunctive-guard loops. The annotations list their diagonal atoms, shared unary bounds, and invariant. Counts below summarize the complete automata: reachable locations, transitions, and active clocks. The internal output-shape guarantee is proved in Rocq; the displayed graph and counts are measurements.

support a general runtime or size bound. The runs of this subsection used the former default, Spot level `small`. The tool's default is now the automatic mode of article Section 8.2; with level `low` N8 (positive form) takes 11.6 s instead of 98.3 s, and the total over the 29 completed configurations falls from 152 s to 58 s (one run against the five-run medians); optimized sizes are identical except for F5 and F6 (25 to 27 symbolic transitions). The optimizer, the export and the initial-value check were since made faster by verified rewritings (article Section 8.3), so the timings of the table are those of the earlier implementation.

7. Contract table and related-work scope comparison

8. Generated continuation example

Figure 4 follows an actual current tool-generated automaton from the reset-completion boundary through optimization and export. It does not repeat the upstream MTL translation. In this instance, two optimization rounds remove nine transitions; export adds three explicit transitions while producing the required conjunctive internal shape.

9. Additional evaluation details and tables

9.1. Generation time on deep nests: profile and the Spot-level trade-off

Where the time goes on deep nests. A profile of Spot on the simplified formula of N7 (12.9 s of processor time, 43 MB) attributes 79% of its time

to its BDD library and 16% to the rest of Spot. None of the options of `spot-x(7)` that we tried (implication-based formula simplification, formula splitting, SCC filtering, explicit iteration over valuations, Boolean relabeling, branching postponement) changed the time beyond the run-to-run spread, and compositional suspension doubled it. On this input the cost may lie in the formula that the verified translation hands to Spot: a bounded eventuality repeats the translation of its operand three times, so the formula has size $\Theta(3^k)$ in the nesting depth k (the proposition p_7 occurs $2{,}187 = 3^7$ times in the formula of N7). Spot's time grows by a factor of roughly 4 to 12 per level on the levels measured while its output (129 states and 4,623 edges at seven levels) grows by about 3.5. On N7, Spot alone (10–13 s) takes about as long as CASAAL's whole translation (11.3 s), and the rest of our chain takes about 5 s. On N7 the difference therefore appears to lie in the replaceable front end (article Section 7.1) and not in the verified optimization and export; this rests on one profile, and we did not profile N8.

Why the default is a trade-off. Against the better explicit setting, automatic mode loses 0.6 s on N5 (where `low` is simply the slower setting), 5.3 s on N6 and 15.6 s on N7 (the 5 s of the abandoned `low` attempt plus run-to-run variation that we did not separate), and it avoids the minutes that `low` alone needs on N7 (393 s in a separate run) and the more than 900 s it needs on N8. On the conjunctions it is never worse than the former default `small`, which needs 41 s on seven responses and fails on eight. On the *positive* forms, which one would give to a model checker as an assumption, automatic mode costs little: seven conjoined bounded responses take 14.0 s with `low`, 8.7 s with `any` and 14.3 s in automatic mode, and eight take 89.9 s in automatic mode, where neither explicit setting finished within the 30 s limit of the earlier comparison; CASAAL's own times on the positive forms were not measured. On mixed formulas, nested chains conjoined with plain responses (Section 8.2), `any` is at worst 7.7 s where `low` needs 0.3 s, which automatic mode avoids by trying `low` first, and `low` exceeds 30 s on nested conjunctions of depth five where `any` needs 0.2 s, which automatic mode reaches after its budget.

9.2. Generation time: earlier runs

Table 14 gives the earlier timing runs (single runs of ours, best of three of CASAAL), with the manual setting where it helps. R, N and F are the benchmark groups defined in article Section 9; N8 under the manual setting took 243 s against 73.8 s for CASAAL.

9.3. TChecker stored states per product

Article Section 9.2.3 describes the setup and the totals; this is the full table.

9.4. Positive-form workload

9.5. Zone graph of the stress workload

Article Section 9.2.4 reports the zone graph of the products of Table 3. This section gives the data. The automata are ours and CASAAL's, the latter converted to location acceptance and exported by our tool with `-n 0` and the all-copy policy.

9.6. A probe of location merging

The conclusion lists a pass that merges equivalent locations as future work. As a lower bound on what it could gain on the benchmarks we computed, on the exported negated automaton, the coarsest syntactic bisimulation (equal accepting mark and invariant, and equal outgoing (event, guard, resets, class of target)); it is sound for the timed language but not complete for timed bisimulation (script `bisim_probe.py`). It merges no location of any of the 23 automata probed (F1–F13 with F3b, R1–R4, N1–N5; TChecker counts exist for 20 of them); it only removes a duplicate edge on F2 (7 to 6 edges) and F12 (17 to 16), and the TChecker counts do not change. The verified optimizer is therefore at the syntactic bisimulation fixed point on these inputs. This says nothing about a semantic (zone-based) merge, which we did not try.

9.7. UPPAAL, model checking and agreement of the verdicts

We generated 311 UPPAAL 5.0.0 models: the negated requirement in seven variants (ours with the default export, with `-n 0` and with the all-copy export; CASAAL's automaton converted to location acceptance, plain and passed through our export in the same three ways), composed with several systems (a single request/response system, parallel request/response components with their own clocks, a sporadic emitter and a free system). The query is reachability of the accepting sink of the negated safety requirement, not Büchi acceptance. All 296 runs with an expected verdict return it, for every variant; the 15 positive-form runs check only that UPPAAL reads the default maximal-delay export, which contains diagonal constraints. On full explorations the number of stored states is identical in all seven variants

on every full exploration except F5 and F8 (3 and 2 states for ours, 4 and 4 for CASAAL's, the smaller translation). Where times differ, they follow the number of explicit property edges: on R6 with parallel components ours needs 1.3 s against 3.5 s for CASAAL (182 edges against 330), and CASAAL's automaton passed through our export is as slow as CASAAL's own (3.6 s), so neither the export nor the optimizer is responsible. Table 20 lists the products on which the times differ. The negated automata of F1–F9, R1–R6 and N1–N3 have no location invariant in any variant. These are checks on small systems and are single reachability queries; they show that the exported models are usable in UPPAAL and give the same verdicts, not a general speedup.

For timed liveness, which UPPAAL does not decide, we reran the separate TChecker case with and without the optimizer on the same system, property automaton and settings. On "responds within 4 units" both runs store 77 states and on "may remain busy forever" both store 15; the exported XML and the product automata are byte-identical within each pair, so these examples show neither a speedup nor a regression. XML printing, the adapter and the format correspondence remain outside the mechanized interface.

9.8. Differential test against an independent judge

To check the automata of both translators against the semantics, not only against each other, we wrote an independent evaluator of the pointwise MTL semantics, transcribed from the Rocq definition of satisfaction, and acceptance evaluators for the UPPAAL XML of our tool (including diagonal guards and invariants) and for CASAAL's DOT output. We drew ultimately periodic timed words with strictly increasing timestamps in multiples of one half, at most nine positions, with delays chosen so that the interval bounds of the formula are hit exactly and one half-unit on either side. On 22 benchmark patterns (20,000 words each) and 100 random formulas (6,000 words each), in both polarities, the automata of our tool (default and all-copy exports) and of CASAAL accept exactly the words that the evaluator accepts (Table 21, 6,160,000 word checks, no disagreement). CASAAL has no pointwise next operator, so F3 and F13 are checked for our tool only. The evaluator itself is tested: it rejects 11 of 13 deliberately wrong formulas for our tool and 7 of 7 for CASAAL. The test finds no bug in either tool; it is agreement on a finite sample of a restricted word class (first event not at time zero, delays multiples of one half), not a proof for either.

9.9. Seeded defects: what testing misses and the proofs reject

The agreement above shows that the tools are correct on the sample, not what the proofs add. To measure that, we seeded 22 plausible defects, one at a time, in a scratch copy of the extracted optimizer and exporter (Table 22; for instance a strict instead of non-strict test at the bound zero, a reset set not compared when two locations are merged, the complement of $x < c$ taken as $x > c$). Each mutant was tested in five ways: T1 compares its output with that of the unmutated tool on the benchmark and the corpus of 148 generated automata; T2 tests, for up to 100,000 random words per input, whether the output automaton accepts the words of its input; T3 (the fuzz) repeats this on 3,000 random automata drawn with the same generator and another seed (a corpus about 20 times larger than the 148, with small constants including zero), testing 5,000 words on each output that differs from the unmutated tool's; T4 compares each such output with its input on every lasso word with a prefix of 0 to 3 and a loop of 1 to 3 positions over all events, with delays in $\{0, 0.5, 1, 2, 5\}$ time units and every initial clock valuation with values in $\{0, 1, 2, 3, 5\}$, which is exact on that finite word set only; and T5 is a structure-directed fuzz on 1,500 further random automata in which each guarded transition receives, with probability one half, a twin with the same source, target, label and resets whose guard is the complement of one atom (for a strict bound, in half of the cases the strict opposite bound, which holds on neither side of $x = c$). Only outputs that differ are tested, because an output byte-identical to the unmutated tool's has the language of the unmutated tool, which the proofs equate with the input's.

Eighteen of the 22 mutants change the language of some automaton; four do not (M05 changes the output but not the language; M08, M21 and M22 change neither). Thirteen changes are found by T2. Five, M01, M10, M12, M16 and M17, are found by neither T1 nor T2: the benchmark and the small corpus never reach the defect (four of the five change no output at all), and the word test, which fails to notice even a single dropped transition in 36% of our controls, does not hit the few words that separate the languages. T3 finds M01, M10, M16 and M17, on 44, 2, 31 and 5 inputs (an automaton under one export policy; 6,000 inputs in all), and T4 the same four, on 18, 1, 27 and 1 of the inputs whose output changes; the one mismatch for M17 is a single word, so M17 does change the language, although T2 alone did not detect it. M12 changes no output on any of the 3,000 automata, so T3 and T4 cannot see it; T5 finds it, with a changed output on 1,020 of 1,500 automata and a language mismatch on 164 (M01, M16, M17 on 26, 16 and

2; M10 on none). For M01 and M12 we built an automaton on which the mutant's language differs (for M01, 852 accepted sample words become none; for M12, 3,234 against 3,564), so M12 is a real language change that every test of this kind missed. The test oracles are themselves unverified code, T4 is exact only on its word set, and no exact language-equivalence check was tried. M12 was found only by a generator written with that pass in mind, which is the point: a test finds a defect when someone thinks of it.

We then applied the same mistakes to the Rocq definitions in a scratch copy of the development (control: the unmutated copy compiles; before the removal of the classical axioms it took about 43 s). All 22 are rejected by Rocq within 1.8 to 9.1 seconds at a named lemma (Table 23). The rejection is at the lemma that ties the implementation to its specification (for M01, against the specification function of the invariants), so the check is only as strong as that specification: a mutation that changed implementation and specification together would pass. It is also conservative: it rejects the four mutants that do not change the language, so a rejection means that the definition no longer matches the specification, not that its language differs. The Rocq mutants are transcriptions of the extracted-code mutants, made by the authors. These results make a bounded claim: for these defects in this code, T1 and T2 missed five of 18 language-changing defects, a corpus about 20 times larger and an exhaustive short-word check missed one, a generator aimed at the merge pass found it, and the development rejects all of them. They do not show that CASAAL contains such defects, and CASAAL may well be correct.

9.10. Export time with and without the optimizer

The maximal-delay export contains diagonal guards from R2 on and TChecker's liveness algorithms reject them, so the all-copy export is the one a TChecker user needs. Without the optimizer it takes 44.0 s on R4 and does not finish on R5 and R6; with it, 0.7, 5.7 and 55.5 s (wall-clock times of the whole run of the tool, single runs). The table does not show a gain in what the checker explores: the stored states are the same with and without the optimizer (Table 16).

9.11. Generated group: setup details and aggregates

This subsection gives the setup details and the aggregates behind article Section 9. Internal output shape is a Rocq theorem; external checker verdicts, sizes and times are experimental evidence. The matched TChecker cases

below show unchanged exploration on systems whose exported models are byte-identical before and after optimization; they do not establish a general cost improvement.

We retain the 31-attempt F/R/N benchmark as an upstream-generated group. The optimizer input is the automaton after reset completion, shown as Comp. in Table 10. Of the 31 attempted configurations, 29 completed this upstream stage and supplied an optimizer input; R7 and R8 timed out before the measured boundary (with the later implementation of article Section 8.3, their positive forms complete in 14 s and 90 s). Formula rewriting, Spot configuration, cube expansion, relaxation, and reset completion happen upstream; their effects are not credited to the optimizer. The runs use the tool's default front end, which before translation rewrites $\Box\Diamond_{\geq d}\varphi$ into $\Box\Diamond\varphi$ and $\Diamond\Box_{\geq d}\varphi$ into $\Diamond\Box\varphi$, and which replaces the Until of the clauses of upper-bounded hatted Until by a weak Until. The end-to-end theorem for this front end is `MTL_to_exported_correct0_recur_weak_with` (`MTL_to_TBA_Recur.v`); the optimizer theorems are unaffected, since they hold for every input TBA of the interface. Results are medians of five runs per completed configuration, with a 300 s limit per attempt, on Ubuntu 26.04/WSL 2, an Intel Core i7-8650U at 1.9 GHz with 11.7 GB of memory, OCaml 5.4.0, Rocq 9.0.1, and Spot 2.16. The generated-group sizes and times of this section were measured with the Spot level `small` and the implementation before the speedups of Section 8.3; the run log, machine record, summary table, and generated automata are preserved under `evaluation/runs/2026-10-14-full-eval-v3/`. The per-instance table is in Table 10.

Across the 29 completed inputs, the sum of reachable transition counts falls from 20,204 at the post-reset-completion boundary to 1,500 after optimization (an aggregate reduction of 92.58%). Nineteen cases have fewer transitions and 10 are unchanged. This aggregate is strongly skewed: R6 contributes 14,985 of the 18,704 removed transitions, so the percentage is not a typical per-instance reduction. Active clocks used in guards or resets sum from 77 to 74; only three cases (F2, F11, F13) reduce this count. The export stage has a different effect: explicit transitions sum from 1,500 to 3,209, an increase of 113.93%; six exports grow and 23 have the same count as their optimized input. Export-size change is reported separately from optimization and is not counted as an optimization regression. The family breakdown shows how concentrated these totals are. The six completed bounded-response R cases account for 18,441 of the 18,704 removed transi-

tions (98.59%); the F and N groups remove 46 and 217, respectively. The R group's explicit export grows from 1,089 to 2,796 transitions, while the F group grows by two and the N group is unchanged. Thus the aggregate optimizer reduction is driven mainly by one bounded-response family, and the export expansion is a separate representation cost concentrated in that same family.

Part of this reduction only re-merges the cubes that Spot emits one by one. With symbolic grouping applied in both runs, the optimizer alone lowers the symbolic transition count (`-n 0` against the default) from 704 to 344 (51%) on the 27 configurations whose unoptimized export completes: from 364 to 216 on F and N, and from 340 to 128 on R1–R4. It is smaller in 17 and equal in 10 (per-instance rows in `evaluation/symbolic_before_after.tsv`). In the recorded runs the unoptimized export of R5 and R6 did not finish within 300 s, whereas the optimized pipeline completed them in 3.6 s and 39.0 s; after the verified speedups of Section 8.3 the unoptimized export of R5 finishes, with 5,469 explicit transitions against 651 for the optimized one.

9.12. Matched comparison of the two export policies

We reran R2–R6 once with each verified split policy while holding the optimized automaton fixed; the optimized TBA DOT files were byte-identical across the two invocations. The table reports explicit-edge counts, diagonal guard atoms and single-run exporter time returned by the tool. All-copy export has no explicit-edge growth beyond the optimized relation and adds no diagonal guards; maximal-delay export adds explicit edges in each case and introduces between 2 and 3,360 diagonal guard atoms per formula (4,464 in total). This is a measured property of this benchmark family, not a general size theorem.

Both policies retain the same reachable location counts, active clock counts and invariant-bearing locations on these runs. Peak RSS of the full tool process, including Spot and optimization, was 17.2/17.3 MB for R2 and 108.5/107.0 MB for R6 (maximal-delay/all-copy); these do not isolate export memory. The observed exporter times have no consistent ordering: all-copy takes longer for R2/R3 and shorter for R4/R5, with similar times for R6.

9.13. Synthetic suite: interaction cases

9.14. Leave-one-out ablation of the optimization passes

Table 28 reports the optimized automaton when one pass of the optimization round is skipped, in a scratch copy of the extracted code (the proved sources and the shipped tool are unchanged; `evaluation/make_ablate_tool.sh` builds the copy and `evaluation/run_pass_ablation.sh` runs it; "guard normalization" skips both normalization steps of the round). The inputs are the requirements as written. The benchmark columns cover the 25 configurations F1–F14, R1–R5 and N1–N5, for which Spot finishes within seconds; the random columns cover the 100 random formulas. "Changed" is the number of configurations whose optimized locations, transitions or clocks differ from those of the full optimizer. The sizes after export are in `evaluation/pass_ablation_summary.tsv`.

10. Random formulas and export policies on every configuration

The 100 random formulas of the translation article (depth at most three, three propositions; `evaluation/random_formulas.tsv`) were run through the default pipeline with the optimizer enabled, by `evaluation/run_random_opt.sh` (current tool, default automatic Spot mode, limit of 120 s per formula); all 100 complete, the slowest in 0.3 s, and none needs more than three optimization rounds. Over the 100 formulas, the explicit transitions of the optimizer input total 3,980 and those of the optimized automaton 2,366; locations 784 and 638; clocks 160 and 131. The optimizer reduces transitions on 50 formulas, locations on 25 and clocks on 25, and increases none of the three. The exported automaton has 638 locations and 2,391 transitions in total (the maximal-delay export adds difference guards on some formulas). The verified `init_free` check succeeds on 96 formulas and fails on four (X38, X65, X84, X100), for which the zero-initial-clock reading of UPPAAL is therefore not covered by the proved condition. A first run with the former default Spot level, whose Spot automata are different, gave 2,507 and 1,509 transitions, 696 and 564 locations, 160 and 128 clocks, reductions on 57, 25 and 26 formulas, and no increase; the qualitative picture is the same. These are size and timing measurements; they do not check language equivalence, which the proofs provide for the optimizer and the export. The per-formula rows are in `evaluation/random_opt_nov9.tsv` (current run) and `evaluation/random_opt.tsv` (first run).

Table 30 compares the two export policies on every configuration of the generated benchmark: the maximal-delay export (default) and the separately verified all-copy split (`-all-copy`), by `evaluation/run_export_policies.sh`. The two exports have the same number of invariants on every completed configuration. The all-copy export has exactly the transitions of the optimized automaton and no difference guards. The maximal-delay export has more transitions on F14 and R2–R6 (for example 1,837 against 640 on R6, with 3,360 difference guards) and the same number elsewhere; the totals over the 29 completed configurations are 3,209 and 1,500. R7 and R8 do not complete under either policy within 300 s.

11. Moved from the article: Spot settings, count sensitivity and non-empty products

This section holds text that the article summarizes: the measurements behind the choice of the Spot level (article Section 8.2), and the checks of how far the verified stages contribute to the TChecker counts and of the verdicts on non-empty products (article Section 9.2.5).

11.1. Choosing the Spot level

The propositional automaton is produced by `ltl2tgba -B -<level> [-x <options>] -hoa` and read back by the tool (the reader is outside the proofs). The theorems hold for every automaton that accepts the propositional models of the clocked formula, so neither level nor options affect correctness; they affect running time a great deal and the size of the result little on our benchmark: levels `small` and `low` give the same optimized size on every configuration but F5 and F6 (25 against 27 transitions).

Two settings matter. Level `low` is instantaneous on conjunctions of bounded responses, where `any` is slow (seven responses, negated: 0.1 s against 11.9 s), and slow on deeply nested bounded eventualities (six levels, negated: 13.9 s against 1.3 s for `any` on the simplified formula). Spot's simulation-based reduction takes more than half of its time at level `any` and removed nothing on our inputs; switching it off with `-x ba-simul=0` cut the time on nested requirements by a factor of 2.8 (seven levels: 28 s to 10 s for Spot alone, same 129 states and 4,623 edges). With that option, `any` and `low` give automata of the same size on each of the 57 pattern–polarity pairs where both finish within the limit.

No level is best everywhere and we found no rule on the shape of the formula (a nested chain of depth five conjoined with five responses takes 0.3 s with `low` and 7.7 s with `any`; without the responses, `low` exceeds 30 s at two conjuncts and `any` needs 0.2 s). The default is therefore an *automatic mode* (`-spot-level auto`): it runs Spot at level `low` and, if Spot has not finished after a budget of b seconds (`-spot-budget`, default 5), stops it and runs level `any` with `ba-simul=0` on the formula simplified by the proved simplification `ltl_simp_correct`. Both steps are instances of the verified translation and only Spot remains trusted. The fallback costs at most b seconds; the choice depends on a wall-clock measurement, so the level, and with it the run time, can differ between runs and machines.

The mode can be overridden with `-spot-level`, `-simp` and `-spot-x`: for requirements whose conjuncts are all deeply nested chains (depth four or more), `-simp -spot-level any -spot-x ba-simul=0` skips the budget spent on `low` (N6: 1.3 s against 6.6 s in automatic mode), and benchmarking should fix the level so that the automaton does not depend on machine speed. On our inputs no other `spot-x` option than the simulation reductions improved the time, and for plain responses no override is worth its cost. article Section 9.2.2 reports automatic and manual times side by side.

11.2. Sensitivity of the TChecker counts

Three checks bound what the verified stages contribute to these counts. *Optimizer.* With the optimizer disabled (`-n 0`), TChecker stores exactly the same number of states on each of the 20 products, although the optimizer reduces the automaton: from 688 to 461 transitions on N5, from 2,737 to 1,541 on N6, from 10,930 to 5,105 on N7 (the size of CASAAL's automata in each case) and, on the positive form of five conjoined bounded responses, the exported automaton from 5,469 to 651 transitions. Running TChecker on the automaton before any pass (the output of reset completion) gives the same count as on ours on six of seven products tried (F1, F7, R2, R3, N2, N3) and 1,790 instead of 1,688 on F12. *Invariants.* Only F10–F12 carry location invariants in our negated automata. Removing them changes the total from 204,564 to 204,749, all of it on F10–F12 (and for CASAAL's automata from 204,819 to 205,004). *Optimizer on CASAAL's automata.* On every CASAAL automaton that the importer converts (F1–F12, R1–R8, N1–N5), the optimizer changes nothing: these automata are a fixed point of our passes, which is consistent with their equal size. A hand-made automaton

with redundant guards shrinks under the same driver, so the importer does not prevent the passes from acting.

11.3. Non-empty products

All products above are empty, so they show the size of the state space but not that the translators agree on a verdict when the system violates the requirement. We therefore composed the negated automata with two kinds of system. A *free* system allows every event at every time (strictly increasing, divergent timestamps): for each of the 20 products, ours (default, `-n 0`, invariants removed) and CASAAL's automaton in three variants (with and without an invariant script, and converted and exported) give a non-empty product, 138 verdicts in all, with no disagreement (F14 cannot be converted). A *response* system for R1–R3 either answers every request within two time units or never answers the first one; the products are empty in the first case and non-empty in the second, for every variant and for both export policies, with no disagreement (42 verdicts for each case). TChecker stops at the first violation, so state counts of non-empty products depend on the search order and are not compared. article Section 9.6 reports the same check with UPPAAL.

References

[1] G. Behrmann, A. David, K. G. Larsen, A tutorial on UPPAAL, in: Formal Methods for the Design of Real-Time Systems, Vol. 3185 of Lecture Notes in Computer Science, Springer, 2004, pp. 200–236. doi:10.1007/978-3-540-30080-9_7.

[2] C. Daws, S. Yovine, Reducing the number of clock variables of timed automata, in: Proceedings of the 17th IEEE Real-Time Systems Symposium, IEEE Computer Society, 1996, pp. 73–81. doi:10.1109/REAL.1996.563702.
URL https://doi.org/10.1109/REAL.1996.563702

[3] S. Guha, C. Narayan, S. Arun-Kumar, Reducing clocks in timed automata while preserving bisimulation, in: CONCUR 2014—Concurrency Theory, Vol. 8704 of Lecture Notes in Computer Science, Springer, 2014, pp. 527–543. doi:10.1007/978-3-662-44584-6_36.

[4] B. Yalcinkaya, E. A. Gol, Clock reduction in timed automata while preserving design parameters, in: Proceedings of the 7th International Workshop on Formal Methods in Software Engineering, 2019, pp. 11–20. doi:10.1109/FormaliSE.2019.00010.

[5] B. Badban, S. Leue, J.-G. Smaus, Automated invariant generation for the verification of real-time systems, in: A. Voronkov, L. Kovacs, N. Bjorner (Eds.), WING 2010: Workshop on Invariant Generation 2010, Vol. 1 of EPiC Series in Computing, EasyChair, 2012, pp. 44–58. doi:10.29007/npn7.
URL https://doi.org/10.29007/npn7

[6] M. D. Wulf, L. Doyen, J.-F. Raskin, Systematic implementation of real-time models, in: FM 2005: Formal Methods, Vol. 3582 of Lecture Notes in Computer Science, Springer, 2005, pp. 139–156. doi:10.1007/11526841_11.

[7] J.-P. Bodeveix, M. Filali, About disjunctive invariants in UPPAAL, in: 15th Nordic Workshop on Programming Theory (NWPT), Turku, Finland, 2003, workshop presentation, 29–31 October 2003.
URL http://www.abo.fi/~Kaisa.Sere/15NWPT.html

[8] S. Wimmer, P. Lammich, Verified model checking of timed automata, in: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2018), Lecture Notes in Computer Science, Springer, 2018, pp. 61–78.

[9] S. Wimmer, Munta: A verified model checker for timed automata, in: Formal Modeling and Analysis of Timed Systems (FORMATS 2019), Lecture Notes in Computer Science, Springer, 2019, pp. 236–243. doi:10.1007/978-3-030-29662-9_14.

[10] S. Wimmer, F. Herbreteau, J. van de Pol, Certifying emptiness of timed Büchi automata, in: Formal Modeling and Analysis of Timed Systems (FORMATS 2020), Vol. 12288 of Lecture Notes in Computer Science, Springer, 2020, pp. 58–75. doi:10.1007/978-3-030-57628-8_4.

[11] N. Saeedloei, F. Kluźniak, Clock allocation in timed automata and graph colouring, in: Proceedings of the 21st International Conference

on Hybrid Systems: Computation and Control (HSCC), 2018, pp. 71–80. doi:10.1145/3178126.3178138.

[12] L. Clemente, R. Mayr, Efficient reduction of nondeterministic automata with application to language inclusion testing, Logical Methods in Computer Science 15 (1) (2019).

Table 8: Parameter sweeps (selected sizes) for five translator-independent synthetic TBA families. The parameter k is defined in the text. A dagger marks a result at the 50-round driver cap. Tuples report declared locations / graph-reachable locations / transitions / active clocks. The optimizer count is the number of structurally changed rounds; optimization time is the mean over 200 in-memory calls. Export diagonals are explicit clock-difference guard occurrences.

Family	k	Input $L/R/E/C$	Opt. $L/R/E/C$	Rounds	Export $L/R/E/C$	Opt. (ms)	Diag.
Duplicate edges	1	2/2/2/0	2/2/2/0	0	4/2/2/0	0.017	0
Duplicate edges	2	2/2/3/0	2/2/2/0	1	4/2/2/0	0.030	0
Duplicate edges	4	2/2/5/0	2/2/2/0	1	4/2/2/0	0.046	0
Duplicate edges	8	2/2/9/0	2/2/2/0	1	4/2/2/0	0.057	0
Duplicate edges	16	2/2/17/0	2/2/2/0	1	4/2/2/0	0.073	0
Duplicate edges	32	2/2/33/0	2/2/2/0	1	4/2/2/0	0.216	0
Unreachable cascade	1	3/1/2/0	3/1/1/0	1	6/1/1/0	0.027	0
Unreachable cascade	2	4/1/3/0	4/1/1/0	2	8/1/1/0	0.075	0
Unreachable cascade	4	6/1/5/0	6/1/1/0	4	12/1/1/0	0.253	0
Unreachable cascade	8	10/1/9/0	10/1/1/0	8	20/1/1/0	1.752	0
Unreachable cascade	16	18/1/17/0	18/1/1/0	16	36/1/1/0	9.324	0
Unreachable cascade	32	34/1/33/0	34/1/1/0	32	68/1/1/0	47.381	0
Unreachable cascade	50	52/1/51/0	52/1/1/0	50†	104/1/1/0	227.747	0
Unreachable cascade	51	53/1/52/0	53/1/2/0	50†	106/1/1/0	239.316	0
Unreachable cascade	64	66/1/65/0	66/1/15/0	50†	132/1/1/0	531.813	0
Redundant bounds	1	2/2/2/1	2/2/2/1	0	4/2/2/1	0.017	0
Redundant bounds	2	2/2/2/1	2/2/2/1	1	4/2/2/1	0.025	0
Redundant bounds	4	2/2/2/1	2/2/2/1	1	4/2/2/1	0.027	0
Redundant bounds	8	2/2/2/1	2/2/2/1	1	4/2/2/1	0.036	0
Redundant bounds	16	2/2/2/1	2/2/2/1	1	4/2/2/1	0.034	0
Synchronous resets	2	2/2/3/2	2/2/3/1	2	4/2/3/1	0.230	0
Synchronous resets	3	2/2/4/3	2/2/4/1	2	4/2/4/1	0.316	0
Synchronous resets	4	2/2/5/4	2/2/4/1	2	4/2/4/1	0.318	0
Synchronous resets	6	2/2/7/6	2/2/5/1	2	4/2/5/1	0.618	0
Disjunctive invariant	2	2/2/3/2	2/2/3/2	0	6/3/10/2	0.030	10
Disjunctive invariant	3	2/2/4/3	2/2/4/3	0	8/4/30/3	0.047	60
Disjunctive invariant	4	2/2/5/4	2/2/4/3	1	8/4/30/3	0.117	60
Disjunctive invariant	6	2/2/7/6	2/2/5/4	1	10/5/68/4	0.213	204

Table 10: Fresh five-run post-processing measurements (13–14 October 2026) for 31 attempted upstream-generated configurations; 29 produced optimizer inputs. Comp. is the automaton after reset completion. L counts graph-reachable locations, E counts transitions from those locations, and C counts active clocks used in guards or resets; n_f is the formula-clock-key count. Opt. and Exp. give reachable $L/E/C$, changed optimization rounds, stage medians, invariant-bearing locations, and diagonal-guard occurrences. R7 and R8 time out upstream, before the optimizer boundary.

ID	Comp. $L/E/C$	n_f	Opt. $L/E/C$	Rnds	Opt. (s)	Exp. $L/E/C$	Inv.	Diag.	Exp. (s)
F1	2/5/1	1	2/5/1	1	< 0.01	2/5/1	1	0	< 0.01
F2	2/5/1	1	1/1/0	1	< 0.01	1/1/0	0	0	< 0.01
F3	2/5/1	1	2/5/1	1	< 0.01	2/5/1	0	0	< 0.01
F3b	1/1/0	1	1/1/0	0	< 0.01	1/1/0	0	0	< 0.01
F4	2/5/1	1	2/5/1	1	< 0.01	2/5/1	1	0	< 0.01
F5	7/38/1	1	7/25/1	1	< 0.01	7/25/1	0	0	0.01
F6	7/38/1	1	7/25/1	1	< 0.01	7/25/1	0	0	< 0.01
F7	2/4/1	1	2/4/1	1	< 0.01	2/4/1	1	0	< 0.01
F8	2/4/0	0	2/4/0	1	< 0.01	2/4/0	0	0	< 0.01
F9	4/9/2	2	4/9/2	1	< 0.01	4/9/2	2	0	< 0.01
F10	5/13/3	3	5/13/3	1	< 0.01	5/13/3	2	0	< 0.01
F11	4/7/2	2	3/4/1	2	< 0.01	3/4/1	1	0	< 0.01
F12	3/9/2	2	2/5/2	2	< 0.01	2/5/2	1	0	< 0.01
F13	3/6/2	2	2/3/1	2	< 0.01	2/3/1	1	0	< 0.01
F14	6/23/2	2	6/17/2	1	< 0.01	6/19/2	4	2	< 0.01
R1	2/5/1	1	2/5/1	1	< 0.01	2/5/1	1	0	< 0.01
R2	4/25/2	2	4/16/2	2	< 0.01	4/19/2	3	2	< 0.01
R3	8/125/3	3	8/44/3	3	0.01	8/67/3	7	30	0.01
R4	16/625/4	4	16/112/4	4	0.06	16/217/4	15	192	0.16
R5	32/3125/5	5	32/272/5	5	0.94	32/651/5	31	880	1.45
R6	64/15625/6	6	64/640/6	6	17.81	64/1837/6	63	3360	10.67
R7	–	–	–	–	–	–	–	–	–
R8	–	–	–	–	–	–	–	–	–
N1	3/5/1	1	3/5/1	1	< 0.01	3/5/1	1	0	< 0.01
N2	4/11/2	2	4/9/2	1	< 0.01	4/9/2	2	0	< 0.01
N3	5/21/3	3	5/16/3	1	< 0.01	5/16/3	3	0	< 0.01
N4	6/36/4	4	6/25/4	1	< 0.01	6/25/4	4	0	0.01
N5	7/57/5	5	7/36/5	1	0.01	7/36/5	5	0	0.02
N6	8/85/6	6	8/49/6	1	0.03	8/49/6	6	0	0.06
N7	9/121/7	7	9/64/7	1	0.10	9/64/7	7	0	0.23
N8	10/166/8	8	10/81/8	1	0.33	10/81/8	8	0	0.70

Table 11: Contract of the verified optimization and export pipeline. The explicit and symbolic outputs preserve the timed-word language of the TBA provided to the Rocq interface.

Aspect	Guaranteed result
Timed-word language	Preserved by every optimization round and by complete export under the development's existential initial-clock semantics.
Explicit guards	Conjunctions of event literals and clock comparisons; export may introduce clock-difference comparisons in entry conditions.
Location invariants	Conjunctions of non-strict clock upper bounds.
Symbolic output	Groups transitions with the same source, target, and set of clocks to reset under Boolean labels; the symbolic automaton also preserves acceptance.
Input adapter	A third-party parser or conversion into the Rocq TBA representation requires a separate correspondence proof to join the verified boundary.

Table 12: Scope comparison with representative prior work. "Not reported" means not stated in the cited source, not that no related result exists.

Prior work	Method and setting	Stated guarantee and mechanization	Difference from this paper
Daws–Yovine [2]	Active-clock reduction and equal-clock merging in timed automata	Algorithmic reduction with case studies; proof-assistant mechanization not reported	Prior clock reduction; no generic TBA pipeline theorem
Guha–Narayan–Arun-Kumar [3]	Minimum-clock timed-bisimilar automaton	Least clock count; doubly exponential construction; mechanization not reported	Stronger minimization objective, not verified passes/export
Yalcinkaya–Aydin Gol [4]	Position-based reduction preserving design parameters	Timed bisimilarity and measured clock reduction; mechanization not reported	Direct clock-reduction prior art; this work verifies a composed pipeline
Badban–Leue–Smaus [5]	CIPM invariant generation and idle-transition pruning; split discussed as a direction	Predicate-abstraction method; no split theorem or mechanization reported	Relevant disjunction problem, not generic TBA split preservation
De Wulf–Doyen–Raskin [6]	AASAP phase copies, watcher product, multirectangular predicates	Safety equivalence for specialized encoding; mechanization not reported	Phase copies, not one-copy-per-disjunct TBA normalization
Bodeveix–Filali [7]	Per-disjunct split of UPPAAL disjunctive invariants	Informal; PVS proof announced, ongoing	Prior splitting; no priority claim
Wimmer–Lammich / MUNTA [8, 9]	Timed-automata reachability checking	Verified checker and executable refinement in Isabelle/HOL	Proves analysis, not TBA optimization/export
Wimmer–Herbreteau–van de Pol [10]	Timed Büchi emptiness certificates	Sound, complete certificate checking in Isabelle/HOL	Verifies checking, not automaton transformation
Saeedloei–Kluźniak [11]	Clock allocation by liveness, interference graph and colouring; optimal in polynomial time for a subclass	Language-preserving allocation on paper; mechanization not reported	Same ingredients; here a Rocq-checked certificate for the Büchi

Table 13: Verified optimization and export passes. Each preservation theorem quantifies over every input TBA under the Rocq timed-word semantics. Definitions and proofs are in Section 1; the passes of Section 3 are not listed.

Pass	Rocq preservation theorem
Upper-bound invariant synthesis; backward and forward propagation; two-sided invariants	`add_invariants_accepts`, `propagate_n_accepts`, `forward_accepts`, `add_two_sided_invariants_accepts`
Removal of contradictory transitions and of the outgoing transitions of locations with no incoming transition	`remove_contradictory_accepts`, `remove_unreachable_accepts`
Removal of dead resets; merging of synchronously reset clocks	`remove_dead_resets_accepts`, `merge_all_accepts`
Normalization of guards; removal of unsatisfiable action labels	`normalize_accepts`, `remove_unsat_accepts`
Merging of transitions; merging of equivalent locations	`merge_trans_accepts`, `merge_states_all_accepts`
Iterated pipeline (n rounds)	`optimize_accepts`
Merging of transitions for export; splitting of disjunctive guards	`merge_transitions_accepts`, `explode_accepts`
Maximal-delay invariant split; all-copy invariant split	`split_accepts`, `split_inv_conjunctive`; `split_all_accepts`, `split_all_inv_conjunctive`
Normalization of exported guards; removal of copies never entered; merging of the initial location	`normalize_d_accepts`, `dprune_n_accepts`, `dmerge_states_all_accepts`
Symbolic transitions	`symbolic_accepts`, `symbolic_ext_accepts`
Complete export	`export_accepts`; shape: `export_guards_conjunctive`, `export_invariants_conjunctive`
All-copy export	`export_all_accepts`; shape: `export_all_guards_conjunctive`, `export_all_invariants_conjunctive`

Table 14: Negated requirements: CASAAL against `mtl2tba`. Size is reachable locations/transitions of the automaton given to the model checker (ours: after optimization). Times are wall-clock seconds for the whole translation (generation plus, for ours, optimization and export), CASAAL on native Windows and `mtl2tba` under WSL 2 on the same machine; CASAAL times are the best of three runs and ours single runs (article Table 1 gives medians of five runs with one driver). Sizes of CASAAL's automata are after conversion to location acceptance. Spot level: the current default (auto, with the fallback marked) and the manual setting. "Auto" is the default of the tool; † marks runs where Spot had not finished within the 5 s budget and the tool fell back to `any` on the simplified formula. The last column gives the time of the fastest explicit setting when it is at least 10% and 0.1 s faster than auto ("–": no explicit setting qualifies, e.g. on N8 the manual run took 243 s against 247.8 s for auto in this table's run): `low`, or `-simp -spot-level any -spot-x ba-simul=0`, written `any`; the sizes of the automata are the same for both settings except where noted in the text.

	CASAAL size	Ours size	CASAAL time	Ours, auto	Ours, manual (setting)
R2	4/8	4/8	0.03	0.1	–
R3	5/11	5/11	0.03	0.1	–
R4	6/14	6/14	0.04	0.1	–
R5	7/17	7/17	0.06	0.1	–
R6	8/20	8/20	0.09	0.1	–
R7	9/23	9/23	0.1	0.1	–
R8	10/26	10/26	0.2	0.1	–
N5	33/461	33/461	0.3	0.8	0.2 (`any`)
N6	65/1541	65/1541	1.9	6.6†	1.3 (`any`)
N7	129/5105	129/5105	11.3	22.1†	15.1 (`any`)
N8	257/16769	257/16769	73.8	247.8†	–

Table 15: States stored by TChecker (`tck-liveness`, `couvscc`) on the product of the negated requirement with a reference system, for the 18 patterns on which every variant completed (F1–F12 with F3b in place of F3, R1–R3, N1–N3). The product is empty, so it is explored completely and the count does not depend on the exploration order. The reference system is the automaton of the requirement itself, exported with the diagonal-free all-copy policy because TChecker rejects diagonal guards. Columns: our optimized automaton, CASAAL's automaton converted to location acceptance and exported by our tool, our automaton with the optimizer disabled (`-n 0`), and with all location invariants removed. F14 (three acceptance sets) is not converted and N4 was not completed by the comparison driver, so neither is included.

Pattern	Ours	CASAAL, converted	Ours, `-n 0`	Ours, no invariants
F1	280	280	280	280
F2	3	3	3	3
F3b	3	3	3	3
F4	196	196	196	196
F5	228	231	228	228
F6	254	257	254	254
F7	280	484	280	280
F8	9	54	9	9
F9	191	191	191	191
F10	1,188	1,188	1,188	1,269
F11	67	67	67	69
F12	1,688	1,688	1,688	1,790
N1	48	48	48	48
N2	967	967	967	967
N3	22,885	22,885	22,885	22,885
R1	280	280	280	280
R2	7,513	7,513	7,513	7,513
R3	168,484	168,484	168,484	168,484
Total	204,564	204,819	204,564	204,749

Table 16: TChecker stored states on the positive forms of the bounded responses R1–R5 composed with a system that violates the requirement on every run (the product is empty), all-copy export of the reference. "Ours, no invariants" deletes the location invariants of our automaton; "CASAAL, location acceptance" is CASAAL's automaton converted to location acceptance and passed through our export ("–": not completed). The positive form is the requirement as written, not its negation.

Requirement	ours	ours, no invariants	CASAAL, location acceptance
R1	48	52	48
R2	48	52	375
R3	48	52	3,439
R4	48	52	34,107
R5	48	52	–

Table 17: TChecker stored states on the positive forms of R1–R5 composed with a system that exercises all n responses and violates the requirement on every run: the requests of the responses $2, \ldots, n$ are made once each and answered in time, then the request of the first response is made and never answered (the product is empty). All-copy export. Columns: ours (default), ours with the optimizer disabled, ours with the invariants removed, CASAAL's automaton with location acceptance without and with our export, and the ratio of the last column to ours ("–": not completed within 180 s, or not generated).

	ours	ours, `-n 0`	ours, no inv.	CASAAL	CASAAL, our export	ratio
R1	48	48	52	52	48	1.0
R2	70	70	74	986	890	13
R3	92	92	96	50,974	47,008	511
R4	114	114	118	2,000,526	1,736,096	15,229
R5	136	–	140	–	–	–

Table 18: Structure of the positive automata as TChecker reads them (all-copy export, events expanded): locations, clocks, explicit edges, and edges for each pair of a location and an event. Both automata are complete: they have an edge for every location and event (20, 56, 144 and 352 pairs on R2–R5), so the edge counts differ because CASAAL's automaton has several overlapping guarded edges for a pair where ours has one.

	locations	clocks	edges, ours	edges, CASAAL	per pair, ours	per pair, CASAAL
R2	4	2	24	51	1.2	2.5
R3	8	3	68	216	1.2	3.9
R4	16	4	176	837	1.2	5.8
R5	32	5	432	3,078	1.2	8.7

Table 19: Zone graph of the products of Table 3 and two ablations (R2, R3; ours/CASAAL). "Dead clocks reset" resets, on every edge, the clocks that are dead at its target. "Differences dropped" counts the distinct zones of a location vector after removing every constraint between two clocks. The Couvreur-based count is 70/890 and 92/47,008. Plain reachability without subsumption (`tck-reach -a reach`) did not terminate on these products.

	R2 ours	R2 CASAAL	R3 ours	R3 CASAAL
location vectors	10	25	16	85
largest number of zones for one vector	17	240	17	7,444
mean out-degree of the zone graph	1.46	1.56	1.54	2.28
stored with dead clocks reset	70	890	92	47,008
distinct zones, differences dropped	70	883	92	40,542
covreach	10	28	16	103

Table 20: UPPAAL 5.0.0, breadth-first search with default extrapolation, on the negated requirement composed with a system, for selected products (par-ok: parallel request/response components that always answer; free: the free system). Of the seven variants, the table shows ours with the default export, ours with the all-copy export, CASAAL converted to location acceptance, and CASAAL converted and passed through our export; the other three are the same automata with the optimizer disabled. Stored states are identical in all seven variants on these products (and on all full explorations except F5 and F8, where ours stores 3 and 2 states against 4 and 4 for CASAAL); times are medians of three runs including process start; the last column gives the number of explicit property transitions of ours and of CASAAL's automaton.

Req.	System	Stored	Time (s)				Prop. edges
		(all variants)	ours	ours all-copy	CASAAL	CASAAL+export	ours/CASAAL
R3	par-ok	49	0.088	0.084	0.084	0.088	56/93
R4	par-ok	261	0.093	0.098	0.100	0.105	90/156
R5	par-ok	1631	0.166	0.171	0.251	0.207	132/235
R6	par-ok	11743	1.277	1.520	3.473	3.629	182/330
N4	free	17	0.105	0.098	0.119	0.124	356/549
N5	free	33	0.188	0.195	0.285	0.278	1275/2063

Table 21: Agreement of the automata with an independent evaluator of the MTL semantics on random ultimately periodic timed words (both polarities of each formula). A word check is one (formula, word, polarity) triple for one automaton; the words are ultimately periodic with delays that are multiples of one half and at most nine positions. CASAAL is checked on the 20 patterns it encodes (it has no next operator for F3 and F13). A disagreement is a word on which the automaton and the evaluator differ.

Set	Automaton under test	Formulas	Word checks	Disagreements
22 benchmark patterns, 20,000 words each	mtl2tba (default export)	22	880,000	0
	mtl2tba (all-copy export)	22	880,000	0
	CASAAL	20	800,000	0
100 random formulas, 6,000 words each	mtl2tba (default export)	100	1,200,000	0
	mtl2tba (all-copy export)	100	1,200,000	0
	CASAAL	100	1,200,000	0

Table 22: Seeded defects in the extracted optimizer and exporter. T1: number of the 380 runs (296 on the corpus of 148 generated automata under two export policies, 84 on the benchmark patterns) whose output differs from the unmutated tool. T2: index (from zero) of the first random word on which the language test finds a difference; a dash means none was found. Fuzz: inputs on which the language test finds a mismatch, over inputs on which the mutant's output differs from the unmutated tool's, an input is one of 3,000 random automata under one export policy (6,000 inputs in all; 5,000 words per output). Outcome: "caught" by T2; "missed by T1, T2" but found by the fuzz; "missed by all tests" (M12, for which a hand-built automaton shows the language change); "no change found" (M05 is a different composition of verified passes, so its language is preserved by the pass theorems; M08, M21 and M22 are argued equivalent). Rocq: all 22 transcriptions are rejected (Table 23).

Mutant	Seeded defect (pass)	T1	T2 first word	Fuzz	Outcome of the tests	Rocq
M01	closed bound 0 on a just-reset clock	0	—	44/234	missed by T1, T2	rejected
M02	invariant from min, not max, of branch bounds	30	44	154/428	caught	rejected
M03	$x \geq 2$, $x \leq 2$ always empty (strictness ignored)	120	0	1940/3456	caught	rejected
M04	lower-bound tests ignored for clock liveness	104	0	918/1650	caught	rejected
M05	normalization before dead-reset removal omitted	78	—	0/1116	no change found	rejected
M06	guard $x > 0$ treated as trivially true	38	0	770/1284	caught	rejected
M07	clocks merged without equal reset sets	14	0	10/32	caught	rejected
M08	entry into initial location not excluded	0	—	0/4	no change found	rejected
M09	accepting and non-accepting locations merged	6	0	56/116	caught	rejected
M10	resets ignored in simulation test	0	—	2/4	missed by T1, T2	rejected
M11	parallel transitions merged without comparing resets	62	1	184/850	caught	rejected
M12	complement of $x < c$ taken as $x > c$	0	—	0/0	**missed by all tests**	rejected
M13	entry lower bound from max, not min	88	0	1616/2098	caught	rejected
M14	split copies entered the wrong way round	15	0	93/195	caught	rejected
M15	entry condition not updated for resets	40	0	263/462	caught	rejected
M16	reset clock with $x \leq 0$ judged false	0	—	31/82	missed by T1, T2	rejected
M17	implication direction in disjunct pruning	1	—	5/29	missed by T1, T2	rejected
M18	invariants ignored when merging exported locations	8	84	36/328	caught	rejected
M19	initial location pruned as unreachable	376	0	6000/6000	caught	rejected
M20	$x \leq u$ taken to imply $x < w$ for $u \leq w$	24	40	43/220	caught	rejected
M21	safety check of dead-reset pass removed	0	—	0/0	no change found	rejected
M22	weaker bound for conjunctive guard	0	—	0/84	no change found	rejected

Table 23: The same mistakes applied to the Rocq definitions: lemma on which the development first fails to compile, with the time in seconds from the start of the build of the mutated file to the first error (Rocq 9.0.1). The first five rows are the defects that T1 and T2 missed (M12 also escapes the fuzz); the others follow in numerical order. M05, M08, M21 and M22 do not change the language, and are rejected because the proofs name the mutated step or need the removed condition.

Mutant	First failing lemma	Time (s)
M01	`inv_guard_w_eq`	4.7
M10	`sim_spec`	3.8
M12	`compl_cases`	4.8
M16	`const_true_spec`	3.0
M17	`insert_g_holds`	4.1
M02	`max_all_ge`	9.1
M03	`contradictory_bounds_sound`	2.7
M04	`rename_item_holds`	3.2
M05	`optimize_step_accepts`	5.4
M06	`trivial_c_sound`	3.8
M07	`mergeable_b_sound`	2.8
M08	`mergeable_b_sound`	3.3
M09	`states_mergeable_fast_eq`	4.3
M11	`same_ends_spec`	4.6
M13	`min_all_le`	1.8
M14	`notlonger_complete`	2.5
M15	`split_trans_in`	3.1
M18	`dmerge_states_ext`	4.7
M19	`dprune_accepts`	7.7
M20	`implies_c_sound`	2.2
M21	`remove_dead_resets_clock_fast_eq`	3.5
M22	`ub_guard_sound`	1.9

Table 24: Explicit edges and export time (seconds) of the positive form of Rn, the conjunction of n bounded responses with the same bound, with the optimizer disabled ($-n\,0$) and with the default number of rounds, for both export policies. One run each of the current build under WSL 2 with a limit of 300 s; a dash is a run that did not finish within the limit. Results: `evaluation/export_scalability.tsv`.

Req.	Maximal-delay export				All-copy export			
	$-n\,0$		default		$-n\,0$		default	
	edges	s	edges	s	edges	s	edges	s
R2	28	0.3	19	0.1	25	0.2	16	0.2
R3	163	0.3	67	0.1	125	0.9	44	0.3
R4	950	0.4	217	0.3	625	44.0	112	0.7
R5	5,469	2.6	651	1.1	–	–	272	5.7
R6	31,008	50.9	1,837	7.9	–	–	640	55.5

Table 25: Family aggregates derived from the completed rows of Table 10. Edge totals are measured at the post-reset-completion optimizer boundary, after optimization, and after explicit export.

Group	n	Input $\sum E$	Opt. $\sum E$	Reduction	Exp. $\sum E$	Cases $\downarrow / = / \uparrow$
F group	15	172	126	26.74%	128	7/8/0
bounded-response group	R 6	19,530	1,089	94.42%	2,796	5/1/0
N group	8	502	285	43.23%	285	7/1/0

Table 26: Matched maximal-delay and all-copy exports for R2–R6. The two policies receive byte-identical optimized TBA graphs. Export time is one observed run per row and policy; it is descriptive and should not be read as a stable speed ranking.

ID	Optimized E	Max-delay E	All-copy E	Max-delay diagonal atoms	All-copy diagonal atoms	Max-delay time (s)	All-copy time (s)
R2	16	19	16	2	0	0.002	0.005
R3	44	67	44	30	0	0.014	0.024
R4	112	217	112	192	0	0.195	0.190
R5	272	651	272	880	0	1.319	1.052
R6	640	1,837	640	3,360	0	9.689	9.614

Table 27: Measured direct TBA examples targeting propagation and pass interactions. Transition and clock counts are total records and active clocks. Leave-one-operation-out variants are experimental composites, not separately verified transformations.

Case (record in Table 5)	Input E/C	Full E/C	Rounds (full)	Measured target-pass effect when omitted
Backward bound tightening	2/1	2/1	1	Incoming guard tightens from $x \leq 5$ to $x \leq 2$; omitting backward synthesis leaves $x \leq 5$.
Forward lower-bound propagation	3/1	3/1	1	Entry fact $x \geq 2$ is added to the outgoing q edge; edge count is unchanged.
Forward contradiction cascade	7/1	4/1	2	Omitting forward propagation or contradiction removal retains 7 edges; the full run removes the $x \leq 1$ branch and then prunes its unentered chain.
Backward contradiction cascade	5/1	1/0	2	Omitting backward synthesis leaves 2 edges/1 clock; omitting contradiction removal leaves 5/1.
Normalization and dead reset	2/1	2/0	1	Normalizing tautological $x \geq 0$ enables reset removal; omitting either pass leaves 1 active clock.
Synchronous-clock contradiction	5/2	3/1	2	Merging the synchronously reset clocks exposes $x \geq 5 \wedge x \leq 3$; omitting clock merge or contradiction removal leaves 5 edges/2 clocks.

Table 28: Optimized sizes with one pass skipped: transitions (Tr), locations (Loc), clocks (Clk), and the number of configurations that change.

	Benchmark (25)				Random (100)			
Skipped pass	Tr	Loc	Clk	Changed	Tr	Loc	Clk	Changed
none (full optimizer)	670	135	47	0	2,366	638	131	0
invariant propagation	670	135	47	0	2,366	638	131	0
forward bound propagation	670	135	47	0	2,366	638	131	0
contradictory guards	670	135	47	0	2,371	640	131	1
unsatisfiable guards	4,183	136	48	12	3,579	773	143	22
unreachable locations	670	135	47	0	2,375	641	132	2
guard normalization	670	135	48	1	2,381	644	151	19
dead resets	684	135	49	4	2,382	643	152	20
clock merging	670	135	49	2	2,371	640	134	3
transition merging	695	135	47	5	2,695	638	131	28
location merging	680	138	47	3	2,410	661	131	8

Table 29: Zone graph of the products of Table 3 as TChecker computes it: states stored by the Couvreur-based algorithm, location vectors among them, largest number of zones for one vector, and states stored by reachability with inclusion subsumption ("covering", `tck-reach -a covreach`). The graphs of R4 were not extracted ("–"; its Couvreur counts are in Table 3).

	ours				CASAAL			
	states	vectors	max zones	covering	states	vectors	max zones	covering
R1	48	4	17	4	48	4	17	4
R2	70	10	17	10	890	25	240	28
R3	92	16	17	16	47,008	85	7,444	103
R4	–	–	–	22	–	–	–	304

Table 30: Both export policies on every configuration. Opt.: transitions of the optimized automaton. Max.-delay (default) and all-copy: transitions, invariants, and difference guards of the exported automaton. R7 and R8 do not complete within 300 s.

ID	Opt.	Max.-delay			All-copy		
		Trans.	Inv.	Diff.	Trans.	Inv.	Diff.
F1	5	5	1	0	5	1	0
F2	1	1	0	0	1	0	0
F3	5	5	0	0	5	0	0
F3b	1	1	0	0	1	0	0
F4	5	5	1	0	5	1	0
F5	25	25	0	0	25	0	0
F6	25	25	0	0	25	0	0
F7	4	4	1	0	4	1	0
F8	4	4	0	0	4	0	0
F9	9	9	2	0	9	2	0
F10	13	13	2	0	13	2	0
F11	4	4	1	0	4	1	0
F12	5	5	1	0	5	1	0
F13	3	3	1	0	3	1	0
F14	17	19	4	2	17	4	0
R1	5	5	1	0	5	1	0
R2	16	19	3	2	16	3	0
R3	44	67	7	30	44	7	0
R4	112	217	15	192	112	15	0
R5	272	651	31	880	272	31	0
R6	640	1837	63	3360	640	63	0
R7	–	–	–	–	–	–	–
R8	–	–	–	–	–	–	–
N1	5	5	1	0	5	1	0
N2	9	9	2	0	9	2	0
N3	16	16	3	0	16	3	0
N4	25	25	4	0	25	4	0
N5	36	36	5	0	36	5	0
N6	49	49	6	0	49	6	0
N7	64	64	7	0	64	7	0
N8	81	81	8	0	81	8	0